

Bản đồ neural: NeRF và 3D Gaussian Splatting (Neural Map Representations)

PHỤ THUỘC C/16/03-nerf-3dgs-va-hop-nhat (cơ chế dựng ảnh thể tích, rasterize Gaussian, và vì sao PSNR không chứng minh hình học — chương này giả định bạn đã nắm phần đó); F/27 (khung bốn cổng để quyết định thay một khối cổ điển); E/24 (thiết kế ablation, oracle experiment); F/31 nếu bạn muốn hiểu bộ tám vết mà nhiều hệ trong chương này mượn lại.

TÓM TẮT

Chương này hỏi một câu duy nhất: có nên thay bản đồ điểm thưa của hệ SLAM hiện tại bằng một bản đồ neural. Bạn sẽ đọc được hai dòng công trình — NeRF và 3D Gaussian Splatting — theo đúng nút thắt mà mỗi mốc gỡ, chứ không theo thứ tự thời gian. Phần nặng nhất là phần đánh giá, vì đây là chủ đề mà chất lượng ảnh và độ chính xác quỹ đạo bị trộn lẫn nhiều nhất. Đọc xong bạn tách được hai trục đó, chỉ ra được điều kiện thí nghiệm ẩn sau một bảng kết quả đẹp, và lập được ngân sách bộ nhớ cùng băng thông cho Jetson Orin dưới dạng công thức, trước khi thử. Nếu chỉ nhớ một điều: một bản đồ đẹp ảnh không phải một bản đồ đúng hình học, và cho tới nay chưa có bằng chứng nào nói rằng bản đồ neural giúp *quỹ đạo* của một hệ stereo-inertial đã tinh chỉnh kỹ tốt lên.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ điểm thưa** — bản đồ của ORB-SLAM3 và họ hàng: vài chục nghìn tới vài trăm nghìn điểm 3D, mỗi điểm kèm một bộ mô tả để nhận lại. Nó đủ để trả lời “tôi đang ở đâu” và không trả lời được “chỗ kia có trống không”.
- **bản đồ dày, TSDF, lưới chiếm dụng** — ba cách nói về cùng một ý: bản đồ gán một giá trị cho *mọi* vị trí trong không gian, không chỉ ở vài điểm đặc trưng. TSDF lưu ở mỗi ô một khoảng cách có dấu tới bề mặt gần nhất — âm ở trong vật, dương ở ngoài, bằng không đúng trên mặt — nên bề mặt có định nghĩa sạch. Lưới chiếm dụng lưu xác suất mỗi ô bị chiếm, và đó là thứ bộ lập kế hoạch đường đi thật sự cần.
- **dựng ảnh ở góc nhìn mới** — cho một số ảnh đã chụp, tạo ra ảnh của cùng cảnh đó từ một vị trí camera chưa từng chụp. Đây là bài toán mà NeRF và 3DGS sinh ra để giải.
- **PSNR, SSIM, LPIPS** — ba thước đo “ảnh dựng ra giống ảnh thật tới đâu”. PSNR đo sai lệch bình phương theo từng điểm ảnh, SSIM đo tương đồng cấu trúc cục bộ, LPIPS đo khoảng cách trong không gian đặc trưng của một mạng đã huấn luyện. Cả ba đều nói về *ảnh*, không nói gì về hình học.
- **ATE và RPE** — hai thước đo quỹ đạo. ATE là sai số tuyệt đối của cả quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số của các chuyển vị tương đối trong một cửa sổ thời gian. Đây là hai con số quyết định một hệ SLAM có dùng được không.
- **bám vết theo mô hình** — cách ước lượng tư thế bằng cách dựng ảnh từ bản đồ hiện có rồi tối ưu tư thế cho ảnh dựng khớp ảnh thật, thay vì ghép đặc trưng giữa hai khung hình. Nó chính xác khi bản đồ đúng và trôi theo bản đồ khi bản đồ sai.
- **quên thăm khốc** — hiện tượng một mạng học dữ liệu mới rồi mất khả năng tái tạo dữ liệu cũ, vì cùng bộ trọng số phải gánh cả hai. Trong bản đồ neural, nó hiện ra thành “đi sang phòng bên rồi quay lại thì phòng cũ mờ đi”.

- **Replica, TUM RGB-D, ScanNet** — ba tập dữ liệu mà gần như mọi bài trong chương này báo cáo trên đó. Replica là cảnh tổng hợp có độ sâu hoàn hảo; TUM RGB-D và ScanNet là cảnh trong nhà quay bằng cảm biến thật, chuỗi ngắn, tốc độ đi bộ.

CÂU HỎI MỞ ĐẦU

- Q1. Bản đồ của ORB-SLAM3 chỉ có vài trăm nghìn điểm rời rạc, mà hệ vẫn định vị chính xác tới centimét. Vậy chính xác thì một bản đồ dày cho bạn thêm *điều gì* mà bạn đang thiếu?
- Q2. Nếu bạn thật sự cần bản đồ dày, vì sao lại là bản đồ neural chứ không phải TSDF — thứ đã chạy thời gian thực từ 2011 và cho bề mặt có định nghĩa sạch?
- Q3. Một hệ dựng ảnh đẹp tới mức khó phân biệt với ảnh thật. Vì sao điều đó vẫn không nói gì về việc quỹ đạo của nó tốt hay tệ hơn hệ bạn đang có?
- Q4. 3DGS nhanh hơn NeRF nhiều bậc độ lớn. Khoản chênh đó đến từ một thay đổi thuật toán duy nhất — nó là gì, và vì sao nó lại đúng lúc mở ra khả năng sửa bản đồ sau khi đóng vòng?
- Q5. Một triệu Gaussian nghe không nhiều. Nhưng khi đưa lên Orin và chạy vài chục vòng tối ưu mỗi khung, cái gì cản trước — dung lượng bộ nhớ hay băng thông bộ nhớ — và vì sao câu trả lời đó làm mọi mẹo tăng tốc suy luận trở nên vô dụng?

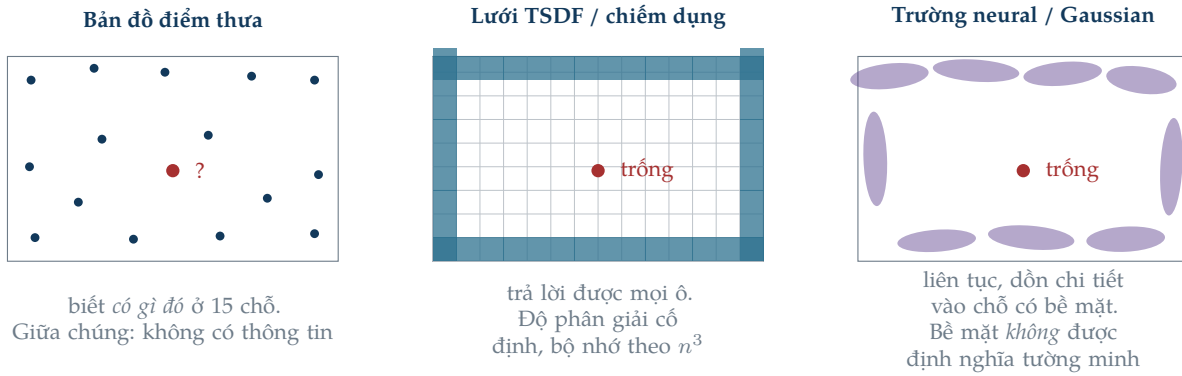
1.1 Đặt vấn đề: bản đồ thưa đủ cho việc gì

Hệ SLAM bạn đang bảo trì dựng một **bản đồ điểm thưa** (*sparse point map*). Mỗi điểm là một vị trí 3D cộng một bộ mô tả nhị phân. Với một tầng văn phòng, con số thường rơi vào bậc 10^5 điểm. Bản đồ đó làm rất tốt đúng một việc: trả lời câu “tôi đang ở đâu”. Nó làm tốt vì nó chỉ cần các điểm *nhận lại được*, và những điểm nhận lại được thì hiếm — góc, biên, đốm vân.

Nhưng có ba câu hỏi hạ nguồn mà chính bản đồ ấy không trả lời nổi. Thứ nhất: khoảng không giữa hai điểm là tường hay là lối đi? Bản đồ thưa không nói. Thứ hai: bề mặt này hình dạng ra sao, để robot đặt tay hoặc để người kiểm tra vết nứt? Bản đồ thưa không nói. Thứ ba: cảnh nhìn từ chỗ camera chưa từng đi qua thì trông thế nào, để người điều khiển từ xa xem trước, hoặc để sinh dữ liệu huấn luyện? Bản đồ thưa cũng không nói.

Trước khi đi tiếp, một điều chỉnh cần thiết cho đúng hệ của bạn. Một cặp stereo đã cho độ sâu dày ở *mỗi* khung hình. Vậy thứ bạn thiếu không phải độ sâu, mà là một bản đồ dày *bền vững*: một chỗ để hợp nhất hàng nghìn quan sát độ sâu nhiều thành một mô hình duy nhất, nhất quán, và tra cứu được ở mọi vị trí. Phân biệt này quan trọng vì nó thu hẹp câu hỏi rất nhiều. Nếu bộ lập kế hoạch của bạn chỉ cần nhìn một mét phía trước, độ sâu stereo tức thời đã đủ và bạn không cần chương này. Nhu cầu bản đồ dày chỉ thành thật khi robot phải nhớ chỗ nó đã đi qua.

Hình 1.1 đặt ba biểu diễn cạnh nhau và hỏi chúng cùng một câu. Đây là hình đáng nhìn kỹ nhất của phần đặt vấn đề, vì nó cho thấy khác biệt không nằm ở “chi tiết hơn” mà nằm ở *loại câu hỏi trả lời được*.



Hình 1.1. Cùng một hành lang, ba biểu diễn, và cùng một câu hỏi hỏi vào điểm đỏ ở giữa: chỗ này trống hay đặc? Bản đồ thưa không có câu trả lời — không phải vì thiếu chi tiết mà vì nó chưa bao giờ mô hình hoá khoảng không. Lưới trả lời được mọi chỗ nhưng trả bằng bộ nhớ tăng theo lũy thừa ba của độ phân giải. Biểu diễn neural trả lời được mọi chỗ và dồn tham số vào nơi có bề mặt, đổi lại bề mặt chỉ tồn tại ngầm dưới dạng một vùng mật độ cao.

TRỰC GIÁC

Bản đồ thưa giống một mạng lưới mốc trắc địa đóng trên nền đất. Đứng giữa công trường, bạn nắm ba cái mốc là biết chính xác mình ở đâu, tới từng centimét. Nhưng không ai đưa bản vẽ mốc trắc địa cho người lái xe cầu, vì nó không nói cái cột nào đang chắn đường. Bản đồ dày là bản vẽ mặt bằng: kém chính xác hơn ở từng điểm, nhưng nói được chỗ nào đi lọt. Hai thứ trả lời hai câu hỏi khác nhau, và một hệ SLAM tốt cho việc định vị hoàn toàn có thể vô dụng cho việc điều hướng.

A. Đối thủ thật của bản đồ neural không phải bản đồ thưa

Đây là chỗ phần lớn bài báo trong chương này lạng lẽ bỏ qua, và cũng là cổng đầu tiên bạn nên dựng. Nhu cầu **bản đồ dày** (*dense map*) có từ lâu và đã có lời giải cổ điển. KinectFusion hợp nhất độ sâu vào một lưới **trường khoảng cách có dấu** (*truncated signed distance field*), viết tắt TSDF, theo thời gian thực từ 2011 [1]. OctoMap cho **lưới chiếm dụng** (*occupancy grid*) dạng cây tám nhánh, tiết kiệm bộ nhớ, có mô hình xác suất rõ ràng, và đã nằm trong hệ điều hướng của rất nhiều robot [2]. Cả hai đều chạy được trên phần cứng nhúng, đều cho bề mặt hoặc thể tích trống có định nghĩa sạch, và đều đã được kiểm chứng hàng chục năm.

Vậy câu hỏi đúng không phải “thưa hay neural”. Nó là: bản đồ neural cho thêm gì so với TSDF, và đổi lại mất gì. Câu trả lời trung thực gồm ba khoản được và ba khoản mất, và cả sáu khoản đều nên nằm sẵn trong đầu bạn trước khi đọc bất kỳ bảng kết quả nào.

- **Được — nội suy chỗ chưa quan sát.** Lưới TSDF để trống ô nào chưa có tia đi qua. Trường neural, do là một hàm liên tục có thiên lệch về hàm trơn, tự lấp các khe nhỏ. Với cảnh quét thưa, khác biệt này rất dễ thấy.
- **Được — dựng được ảnh.** TSDF cho hình học nhưng không cho màu phụ thuộc góc nhìn. Muốn ảnh dựng ra giống ảnh thật, bạn cần một mô hình bức xạ, và đó chính là thứ neural rendering mang lại.
- **Được — tham số dồn theo nội dung.** Lưới đều tiêu bộ nhớ cho cả không khí. Trường neural và tập Gaussian tiêu tham số ở nơi có bề mặt.
- **Mất — bề mặt không tường minh.** Trong TSDF, bề mặt là tập mức 0. Trong trường mật độ hoặc tập Gaussian, không có định nghĩa nào tương đương, nên mọi thứ hạ nguồn cần bề mặt đều phải thêm một bước dựng lại.

- **Mất — không có mô hình xác suất chiếm dụng.** OctoMap nói “ô này bị chiếm với xác suất 0,8”. Mật độ của NeRF không phải xác suất chiếm dụng, và độ đục của một Gaussian càng không. Bộ lập kế hoạch nào cần biên an toàn sẽ không có thứ để dựa vào.
- **Mất — chi phí là vòng lặp tối ưu, không phải một lượt tích hợp.** TSDF cập nhật bằng một phép trộn có trọng số, chi phí cố định mỗi khung. Bản đồ neural cập nhật bằng vài chục bước hạ gradient, và chi phí đó không giảm được bằng lượng tử hoá hay TensorRT.

Khoản mất cuối cùng là khoản đắt nhất trên phần cứng nhúng, và mục 1.5 sẽ quy nó thành con số. Trước đó, hai mục tiếp theo đọc hai dòng công trình theo đúng mạch nút thắt.

I 1.2 Dòng NeRF: một MLP làm bản đồ, rồi mọi thứ sinh ra từ chỗ nó hỏng

Cơ chế dựng ảnh thể tích đã nằm ở C/16/03-nerf-3dgs-va-hop-nhat, nên mục này không nhắc lại. Điều đáng học ở đây là chuyện khác: khi mang cơ chế đó vào một hệ SLAM chạy trực tuyến, những giả định nào vỡ, và mỗi công trình tiếp theo vá đúng chỗ nào.

A. iMAP — ý tưởng cực đoan nhất, và nó giải quyết việc gì

iMAP làm đúng một việc táo bạo: dùng *một* perceptron nhiều lớp làm toàn bộ bản đồ của một hệ SLAM RGB-D chạy trực tuyến [3]. Không có lưới, không có point cloud, không có TSDF. Cảnh là trọng số của một mạng nhỏ. **Bám vết theo mô hình** (*frame-to-model tracking*) là tối ưu tư thế khung hình mới sao cho ảnh và độ sâu dựng ra từ mạng khớp với quan sát. Dựng bản đồ là tiếp tục hạ gradient trên trọng số mạng, dùng các tia lấy mẫu từ một tập **khung hình chốt** (*keyframe*).

Nó giải quyết việc gì cụ thể? Một hệ TSDF phải chọn trước độ phân giải ô lưới và phải trả bộ nhớ cho toàn bộ thể tích. iMAP không phải chọn gì cả: sức chứa nằm ở số trọng số, và mạng tự phân bổ chi tiết. Với một căn phòng, tác giả báo cáo mô hình chỉ vài trăm nghìn tham số — nhỏ hơn nhiều bậc so với lưới tương đương. Thêm một điểm thiết kế đáng học: iMAP chọn khung hình chốt theo *lượng thông tin mới*, đo bằng sai số dựng ảnh của khung hình đó dưới bản đồ hiện tại. Đây là một tiêu chí có cơ sở, khác hẳn các luật chốt khung hình bằng ngưỡng số điểm khớp mà hệ cổ điển vẫn dùng.

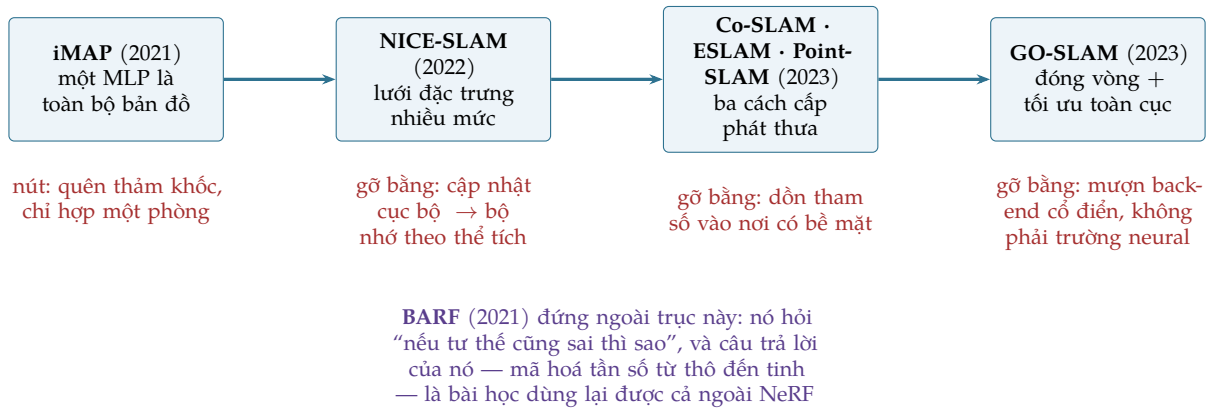
Cụm “vài trăm nghìn tham số” đáng dừng lại một nhịp, vì nó là lý lẽ mạnh nhất của cả dòng này. Điều đáng nhớ không phải một tỉ số cụ thể mà là *hai đại lượng đó lớn lên theo hai kiểu khác nhau*. Sức chứa của một MLP bốn tầng ẩn rộng 256 là một hằng số: nó không đổi khi căn phòng to ra. Bộ nhớ của một lưới thể tích thì tăng theo lũy thừa ba của độ phân giải và tỉ lệ với thể tích, lại còn trả tiền cho cả không khí lẫn tường. Nên ở độ phân giải vài centimet cho một căn phòng, biểu diễn ẩn gọn hơn hẳn — và càng gọn hơn khi bạn đòi độ phân giải mịn hơn. Tỉ số gọn hơn bao nhiêu lần thì phụ thuộc độ phân giải và kích thước phòng, nên nó không phải một hằng số trích dẫn được.

Cái giá thì lộ ra ngay. Vì toàn bộ cảnh nằm trong một bộ trọng số dùng chung, mỗi bước cập nhật ở phòng B đều làm đổi hàm ở phòng A. Đó là **quên thảm khốc** (*catastrophic forgetting*), và cách chữa duy nhất trong khuôn khổ ấy là liên tục phát lại các tia từ khung hình cũ — tức là trả chi phí tăng theo kích thước cảnh cho một biểu diễn vốn được quảng cáo là gọn. Các tác giả công bố hệ trên một GPU để bàn đời cao, với khâu dựng bản đồ chạy chậm hơn hẳn khâu bám vết. Chính chênh lệch đó, chứ không phải con số tốc độ, mới là điều đáng nhớ: khâu bản đồ là khâu bị ngân sách ép, và mọi công trình sau đều tấn công vào đúng nó. Phạm vi thí nghiệm là một căn phòng.

I Mạch 1 — dòng NeRF làm bản đồ SLAM

- 1 Vấn đề** Bản đồ dày cổ điển phải chọn trước độ phân giải và trả bộ nhớ cho cả không khí.
- Giải pháp** iMAP đặt toàn bộ cảnh vào trọng số một MLP, tối ưu bằng chính sai số ảnh và độ sâu [3].
- Giả định** cảnh cỡ một căn phòng, có độ sâu RGB-D, camera đi chậm, và một GPU để bàn luôn sẵn.
- Giá phải trả** mỗi khung hình cần vài chục bước hạ gradient thay vì một lượt tích hợp.
- Vấn đề mới** trọng số dùng chung cho cả cảnh, nên học chỗ mới làm hỏng chỗ cũ.
- 2 Vấn đề** Quên thảm họa, và chi phí phát lại tăng theo kích thước cảnh.
- Giải pháp** NICE-SLAM thay MLP đơn bằng lưới đặc trưng nhiều mức, mỗi bước cập nhật chỉ chạm các ô lưới quanh tia [4].
- Giả định** cảnh có biên giới hạn trước để cấp phát lưới; vẫn cần độ sâu.
- Giá phải trả** bộ nhớ quay lại phụ thuộc thể tích cảnh, đúng thứ iMAP muốn tránh.
- Vấn đề mới** lưới đều lại tiêu tham số cho không khí, và tra nhiều mức thì chậm.
- 3 Vấn đề** Lưới đều tốn bộ nhớ; MLP đơn quên; cần một chỗ đứng giữa.
- Giải pháp** ba lời giải song song — Co-SLAM ghép mã hoá toạ độ với bảng băm thưa [5]; ESLAM thay lưới ba chiều bằng ba mặt phẳng đặc trưng và dùng TSDF thay mật độ [6]; Point-SLAM neo đặc trưng lên chính các điểm đã quan sát, dày thưa theo độ chi tiết ảnh [7].
- Giả định** bề mặt chiếm phần rất nhỏ thể tích, nên cấu trúc thưa là đúng bản chất bài toán.
- Giá phải trả** thêm siêu tham số cấp phát, và mỗi lời giải mạnh trên một loại cảnh khác nhau.
- Vấn đề mới** cả ba vẫn không có tính nhất quán toàn cục: không đóng vòng, không hiệu chỉnh chùm tia toàn phần.
- 4 Vấn đề** Đi một vòng lớn rồi quay lại thì bản đồ lệch, và không có cơ chế nào kéo nó về.
- Giải pháp** GO-SLAM ghép một bộ bám vết dày kiểu DROID với bản đồ neural, thêm đóng vòng và tối ưu toàn cục, rồi biến dạng bản đồ theo tư thế đã sửa [8].
- Giả định** bản đồ neo được vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo.
- Giá phải trả** chi phí tăng thêm một bộ bám vết nặng, và toàn hệ cần bộ nhớ GPU lớn.
- Hệ quả** đây là mốc đầu tiên trong dòng này trả lời được nỗi đau đóng vòng — và nó trả lời bằng cách mượn lại kiến trúc của SLAM cổ điển, không phải bằng bản thân trường neural.

Hình 1.2 vẽ lại chính mạch đó trên một trục, để bạn nhìn thấy mỗi mốc gỡ nút nào.



Hình 1.2. Dòng NeRF làm bản đồ SLAM, đọc theo nút thất chứ không theo năm. Mỗi mũi tên là một câu trả lời cho đúng một chỗ hỏng của mốc trước, và mốc cuối đáng chú ý nhất vì nó gỡ nút bằng cách gọi lại kiến trúc SLAM cổ điển. Nếu bài toán của bạn không có nút thất tương ứng, mốc đó không mang lại gì cho bạn.

B. NICE-SLAM và ý nghĩa của tính cục bộ

NICE-SLAM chữa quên thăm khốc bằng một thay đổi nghe rất kỹ thuật nhưng thực ra là thay đổi khái niệm [4]. Thay vì để cảnh nằm trong trọng số, nó để cảnh nằm trong **lưới đặc trưng nhiều mức** (*multi-level feature grid*): mỗi ô lưới giữ một vectơ đặc trưng học được, còn các bộ giải mã nhỏ thì cố định. Muốn biết mật độ tại một điểm, ta nội suy đặc trưng từ các ô lân cận rồi cho qua bộ giải mã.

Hệ quả quan trọng nằm ở *giá đỡ của gradient*. Với MLP đơn, một tia ở phòng B sinh gradient chảy vào mọi trọng số, nên nó đổi hàm ở khắp nơi. Với lưới đặc trưng, tia đó chỉ chạm vài ô quanh nó. Cập nhật trở thành cục bộ, và quên thăm khốc biến mất gần hết. Hình 1.3 vẽ đúng khác biệt này.



Hình 1.3. Vì sao đổi từ trọng số sang lưới lại chữa được quên thăm khốc. Câu trả lời nằm ở giá đỡ của gradient: bên trái, một quan sát mới chạm tới toàn bộ tham số, nên nó bắt buộc phải đổi cả những vùng cảnh không liên quan; bên phải, quan sát chỉ chạm các ô lân cận. Đây là cùng một nguyên lý với việc chia bản đồ thành nhiều bản đồ con trong SLAM cổ điển: giới hạn phạm vi ảnh hưởng của một quan sát chính là cách mua lấy tính ổn định.

Cái giá thì đúng bằng thứ iMAP muốn tránh: bộ nhớ quay lại phụ thuộc thể tích cảnh, và bạn phải biết trước biên của cảnh để cấp phát. Ba công trình năm 2023 đi tìm chỗ đứng giữa hai cực đó, mỗi cái theo một hướng khác. Co-SLAM ghép một mã hoá tọa độ trơn với một bảng băm thưa, để có cả tính lấp khe của hàm liên tục lẫn tốc độ của bảng tra [5]. ESLAM bỏ lưới ba chiều, thay bằng ba mặt phẳng đặc trưng vuông góc nhau, nên bộ nhớ tăng theo bình phương chứ không theo lũy thừa ba của độ phân giải; nó cũng đổi mật độ lấy TSDF, tức là kéo bề mặt về lại chỗ có định nghĩa sạch [6]. Point-SLAM đi xa nhất theo hướng “chỉ cấp phát nơi có vật”: đặc trưng neo trên chính đám mây điểm đã quan sát, và mật độ điểm được điều chỉnh theo độ biến thiên của ảnh, tức là dồn sức chứa vào chỗ nhiều chi tiết [7].

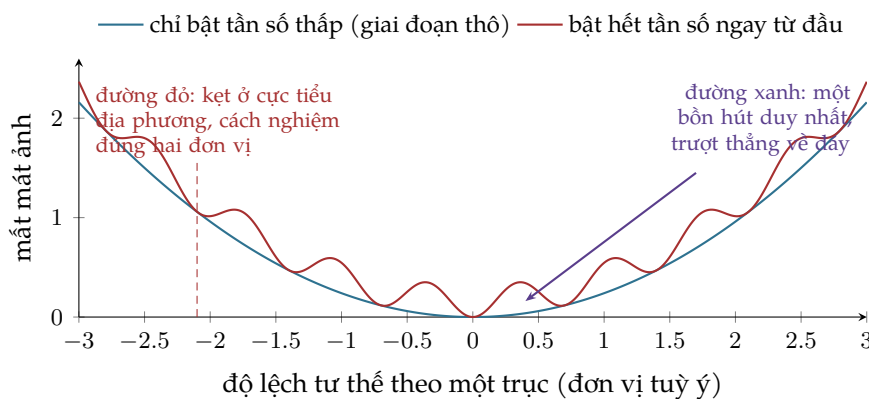
Ba lời giải này đáng nhớ không phải vì tên của chúng mà vì chúng là ba câu trả lời cho cùng một câu hỏi: *đặt tham số ở đâu*. Đó cũng đúng là câu hỏi bạn sẽ phải trả lời nếu tự dựng một bản đồ dày cho hệ của mình.

C. BARF: khi tư thế cũng là ẩn số, và bài học từ thô đến tinh

Cả nhóm trên giả định tư thế camera đã có, hoặc ít nhất có một ước lượng ban đầu tốt. BARF hỏi câu ngược lại: nếu tối ưu tư thế *cùng lúc* với trường thì sao [9]. Về mặt hình thức, đây chính là **hiệu chỉnh chùm tia** (*bundle adjustment*), chỉ khác là điểm 3D được thay bằng một trường liên tục và phần dư được đo trên chính điểm ảnh.

Phát hiện quan trọng của BARF không phải kết quả mà là *nguyên nhân của thất bại*. **Mã hoá vị trí** (*positional encoding*) bằng chuỗi sin và cos nhiều tần số là thứ cho NeRF độ sắc nét. Nhưng chính nó phá hỏng gradient theo tư thế. Lý do là đạo hàm của một thành phần tần số k có biên độ tỉ lệ với k . Các thành phần tần số cao vì thế tạo ra một mặt phẳng mất mát lờm lờm chớm với vô số cực tiểu địa phương cách nhau rất gần. Hạ gradient trên tư thế rơi vào cái hố gần nhất và đứng lại đó.

Lời giải của BARF là che dần: bật các tần số từ thấp lên cao theo tiến trình tối ưu. Giai đoạn đầu, mặt phẳng mất mát trơn và có một bồn hút rộng, nên tư thế đi đúng hướng. Giai đoạn sau, tần số cao mở ra và chi tiết trở lại. Hình 1.4 vẽ minh hoạ mặt cắt một chiều của hai mặt phẳng mất mát đó.



Hình 1.4. Mặt cắt một chiều của mặt phẳng mất mát theo tư thế, hình *minh hoạ* chứ không phải số đo. Đường đỏ là điều xảy ra khi mọi tần số của mã hoá vị trí đều bật: biên độ đạo hàm của thành phần tần số k tỉ lệ với k , nên các gợn tần số cao sinh ra hàng loạt cực tiểu địa phương gần nhau. Đường xanh là cùng bài toán khi chỉ bật tần số thấp: một bồn hút duy nhất, đủ rộng để hạ gradient tìm về đáy. BARF bật dần từ đường xanh sang đường đỏ trong quá trình tối ưu.

Bài học ở đây quan trọng hơn bản thân BARF, và nó nói bằng đúng ngôn ngữ bạn đã quen. Mọi phương pháp căn chỉnh theo cường độ ảnh đều cần một lịch làm trơn. Các hệ trực tiếp cổ điển gọi nó là kim tự tháp ảnh: căn chỉnh ở mức thô trước, rồi mới xuống mức tinh. BARF phát hiện lại đúng nguyên lý đó trong không gian tần số của mã hoá vị trí. Nếu một ngày bạn ghép một hàm mất mát quang trắc vào hệ của mình — dù là để tinh chỉnh tư thế, để căn chỉnh với bản đồ, hay để hiệu chỉnh trực tuyến — thì câu hỏi đầu tiên phải hỏi là: lịch làm trơn của tôi ở đâu.

D. Bốn nhánh còn lại, mỗi nhánh một câu hỏi khác

Bốn công trình còn lại trong danh sách không nằm trên cùng một trục với nhóm trên, vì mỗi cái hỏi một câu riêng.

- **NeRF-SLAM — bỏ độ sâu cảm biến đi thì sao?** Nó ghép một bộ bám vết dày đơn ảnh với một bản đồ kiểu Instant-NGP, và điểm thiết kế đáng học là dùng *độ bất định* của độ sâu ước lượng làm trọng số cho hàm mất mát dựng bản đồ [10]. Đây chính là tư duy ma trận thông tin của đồ thị nhân tử, mang sang phía bản đồ. Nếu bạn chỉ đọc một ý từ cả nhóm này, hãy đọc ý đó.
- **NICER-SLAM — chỉ có RGB thì lấy hình học ở đâu?** Nó thay độ sâu cảm biến bằng các tín hiệu học được: độ sâu đơn ảnh, pháp tuyến bề mặt, và ràng buộc căn chỉnh giữa các khung hình [11]. Nói cách khác, khi mất một ràng buộc vật lý thì phải mượn một tiên nghiệm học được để thay — và mọi giới hạn của tiên nghiệm đó, đặc biệt là vấn đề tỉ lệ, đi theo vào hệ. Chỗ này nổi thẳng sang F/30.
- **vMAP — vì sao cả cảnh phải nằm chung một trường?** Nó cấp cho mỗi vật thể một mạng nhỏ riêng, rồi hợp thành cảnh lúc dựng ảnh [12]. Cái được rất đáng chú ý: vật thể quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật chứ không phải cả căn phòng. Đây là bước bắc cầu sang bản đồ mức đối tượng ở F/33.
- **GO-SLAM — ai kéo bản đồ về khi đóng vòng?** Nó thêm phát hiện vòng lặp và tối ưu toàn cục, rồi biến dạng bản đồ neural theo tư thế đã sửa [8]. Đáng chú ý là cách nó làm được: bản đồ được neo vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo. Bài học chung sẽ trở lại ở mục sau: *biểu diễn nào neo được vào tư thế thì sửa được sau khi đóng vòng*.

E. Giải phẫu hàm mất mát, để đọc bài nhanh hơn

Mọi hệ trong dòng này tối ưu một tổng của vài số hạng, và bốn số hạng dưới đây gần như luôn có mặt. Nhận ra chúng giúp bạn đọc một bài mới trong mười phút thay vì một giờ, vì phần khác biệt giữa các bài thường chỉ là trọng số và một số hạng phụ.

- **Số hạng màu** — sai lệch giữa ảnh dựng và ảnh thật, trên các tia lấy mẫu. Đây là số hạng duy nhất bắt buộc, và cũng là số hạng *không* ràng buộc hình học, đúng như phép tính ở mục sau cho thấy.
- **Số hạng độ sâu** — sai lệch giữa độ sâu dựng ra và độ sâu cảm biến. Đây mới là số hạng thực sự dựng hình học, và nó biến mất trong các hệ chỉ có RGB. Bỏ nó đi thì phải mượn tiên nghiệm học được để thay.
- **Số hạng không gian trống** — ép mật độ về không dọc phần tia trước điểm chạm. Nghe phụ nhưng rất quan trọng: nếu thiếu, mô hình thoải mái để lại vật chất giả lơ lửng, vì không có gì phạt nó.
- **Số hạng chính quy hoá** — ép bề mặt trơn, ép Gaussian dẹt và bám mặt, ép pháp tuyến nhất quán. Đây là chỗ người ta *trả lại* phần ràng buộc hình học đã cố tình bỏ đi lúc đầu.

Một quy tắc đọc rút ra: xem tỉ lệ giữa số hạng độ sâu và số hạng màu. Tỉ lệ nghiêng về màu thì bài đó đang tối ưu cho ảnh đẹp; nghiêng về độ sâu thì đang tối ưu cho hình học đúng. Rất ít bài nói thẳng điều này, nhưng bảng siêu tham số luôn nói.

Bảng 1.1. Dòng NeRF làm bản đồ SLAM. Cột cuối là cột đáng đọc nhất: nó cho biết bạn sẽ gặp gì khi mang hệ đó ra khỏi điều kiện mà nó được công bố.

Hệ	Cơ chế cốt lõi	Giả định	Hỏng khi nào
iMAP	Một MLP là toàn bộ bản đồ; bám vết và dựng bản đồ đều bằng hạ gradient	RGB-D, một phòng, GPU để bàn	Cảnh lớn dần: quên thăm khốc, chi phí phát lại tăng theo diện tích
NICE-SLAM	Lưới đặc trưng nhiều mức, bộ giải mã cố định, cập nhật cục bộ	Biên cảnh biết trước để cập phát lưới	Cảnh lớn hoặc mở: bộ nhớ theo thể tích; không có đóng vòng
Co-SLAM	Mã hoá toạ độ trơn ghép bảng băm thưa	Bề mặt trơn từng mảnh	Va chạm bám ở vùng chi tiết dày
ESLAM	Ba mặt phẳng đặc trưng thay lưới 3D; TSDF thay mật độ	Cảnh không quá nhiều lớp chồng theo mỗi trục	Cảnh phức tạp làm ba mặt phẳng chồng lẫn thông tin
Point-SLAM	Đặc trưng neo trên điểm đã quan sát, dày theo chi tiết ảnh	Độ sâu đủ tốt để đặt điểm đúng chỗ	Độ sâu nhiễu đặt điểm sai; vùng chưa quan sát để trống
NeRF-SLAM	Bám vết dày đơn ảnh + bản đồ băm, trọng số theo bất định độ sâu	Bộ bám vết đủ tốt và chạy được thời gian thực	Bộ bám vết trôi thì bản đồ nhận sai; nặng GPU
NICER-SLAM	Chỉ RGB, mượn độ sâu và pháp tuyến học được làm ràng buộc	Tiên nghiệm đơn ảnh đúng cho loại cảnh này	Ra ngoài phân phối huấn luyện; tỉ lệ trôi
vMAP	Mỗi vật một mạng nhỏ, hợp lúc dựng ảnh	Có bộ tách vật thể đủ ổn định	Tách nhầm vật; cảnh không phân được thành vật
GO-SLAM	Bám vết dày + đóng vòng + tối ưu toàn cục, bản đồ biến dạng theo	Bản đồ neo được vào khung hình chốt	Ngân sách GPU lớn; độ trễ khi biến dạng bản đồ lớn

1.3 Dòng 3D Gaussian Splatting: vì sao rasteri hoá đổi cuộc chơi

Từ 2023 trở đi gần như cả lĩnh vực chuyển sang 3D Gaussian splatting. Chuyện này thường được kể như “nhanh hơn nên tốt hơn”, và cách kể đó bỏ mất phần đáng học. Khoản chênh tốc độ đến từ một thay đổi thuật toán duy nhất, và chính thay đổi ấy kéo theo ba hệ quả rất hợp với SLAM.

A. Khác biệt cơ chế, nói gọn trong một đoạn

NeRF phải *đi tìm* vật chất: với mỗi điểm ảnh, nó lấy hàng chục mẫu dọc tia và chạy mạng ở từng mẫu, vì nó không biết trước mặt nằm ở đâu. 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên nó chỉ cần *rasterize*: chiếu chúng xuống mặt ảnh, sắp theo độ sâu rồi trộn — đúng việc mà phần cứng đồ hoạ sinh ra để làm [13]. Chi phí mỗi điểm ảnh vì thế đổi từ “số mẫu dọc tia” thành “số nguyên thể phủ lên điểm ảnh đó”, và con số thứ hai nhỏ hơn hẳn. Cơ chế đầy đủ nằm ở C/16/03-nerf-3dgs-va-hop-nhat; ở đây ta chỉ cần ba hệ quả của nó.

Chênh lệch nằm ở đâu thì viết ra được mà không cần một con số nào. Chi phí một khung hình của NeRF là (số điểm ảnh) $\times$ (số mẫu dọc mỗi tia) $\times$ (chi phí *một lần chạy mạng*). Của 3DGS là (số điểm ảnh) $\times$ (số nguyên thể phủ lên điểm ảnh đó) $\times$ (chi phí *một phép trộn*). Hai công thức có chung thừa số đầu và khác hẳn ở hai thừa số còn lại: số nguyên thể phủ lên một điểm ảnh nhỏ hơn số mẫu dọc một tia, và một phép trộn số học rẻ hơn một lần chạy mạng rất nhiều. Hai thừa số cùng nhỏ đi, nhân với nhau, là lý do khoảng cách lên tới nhiều bậc độ lớn — và đó là chênh lệch về *bản chất thuật toán*, không phải về cài đặt khéo hay vụng.

- **Cập nhật bản đồ trở thành sửa chữa cục bộ.** Thêm quan sát mới nghĩa là thêm vài nghìn Gaussian ở vùng vừa thấy và tinh chỉnh các Gaussian lân cận. Không có tham số dùng chung toàn cục nào bị chạm. Quên thăm hoa, nút thắt lớn nhất của iMAP, biến mất theo kiến trúc chứ không nhờ mẹo huấn luyện.
- **Bản đồ dịch chuyển được sau khi đóng vòng.** Mỗi Gaussian có một vị trí tường minh. Gắn nó vào khung hình chốt đã sinh ra nó, rồi khi tối ưu toàn cục sửa tư thế khung hình đó, ta dịch chuyển cứng cả cụm Gaussian theo. Với một MLP đơn, phép tương đương không tồn tại: không có cách nào “dịch một phần của trọng số”.
- **Đạo hàm theo tư thế có dạng giải tích.** Phép chiếu Gaussian là một chuỗi phép biến đổi trơn, nên gradient của sai số ảnh theo sáu bậc tự do của camera viết ra được, không cần đi qua mạng. Đây là điều làm bám vết theo mô hình chạy được ở tốc độ dùng được [14].

Hệ quả thứ hai đáng dừng lại một nhịp, vì nó chạm đúng một trong bốn nỗi đau của bạn. Trong hệ hiện tại, đóng vòng sửa tư thế rồi kéo các điểm bản đồ theo, và toàn bộ việc đó rẻ vì bản đồ là một tập điểm rời rạc gắn với khung hình chốt. Một bản đồ Gaussian giữ nguyên tính chất đó. Một bản đồ MLP thì không. Nếu bạn phải chọn một biểu diễn dày cho một hệ có đóng vòng, tiêu chí “sửa được sau khi đóng vòng” quan trọng hơn hẳn tiêu chí PSNR.

I Mạch 2 — dòng Gaussian làm bản đồ SLAM

- 1 Vấn đề** Dòng NeRF chậm ngay cả trên GPU để bản — chậm tới mức khâu dựng bản đồ không theo kịp khâu bám vết — và lại quên chỗ cũ khi đi sang chỗ mới.

Giải pháp thay trường ảnh bằng tập Gaussian tường minh, dựng ảnh bằng raster hoá thay vì lấy mẫu dọc tia [13].

Giả định bề mặt xấp xỉ được bằng các hạt mờ dị hướng, và số hạt phủ lên mỗi điểm ảnh không quá lớn.

Giá phải trả dung lượng bản đồ lớn hơn hẳn một MLP, và từ đây bộ nhớ chứ không phải tốc độ mới là ràng buộc quyết định.

Hệ quả cập nhật thành cục bộ, nên quên thăm khốc biến mất theo kiến trúc.
- 2 Vấn đề** Bám vết cần đạo hàm của sai số ảnh theo tư thế, và đi qua một mạng thì vừa chậm vừa ồn.

Giải pháp MonoGS viết thẳng Jacobian giải tích của phép chiếu Gaussian theo tư thế trên nhóm Lie [14].

Giả định chuyển động giữa hai khung hình đủ nhỏ để nghiệm nằm trong bồn hút quang trắc.

Giá phải trả vẫn là bám vết theo mô hình, nên lỗi bản đồ chảy thẳng vào quỹ đạo.

Vấn đề mới chuyển động nhanh và ảnh mờ đẩy nghiệm ra ngoài bồn hút, và không có gì kéo nó về.
- 3 Vấn đề** Vòng lặp dựng ảnh nằm trên đường ước lượng tư thế nên vừa chậm vừa rủi ro.

Giải pháp GS-ICP tách hẳn hai việc: tư thế do ICP tổng quát trên tâm Gaussian, dựng ảnh chỉ là sản phẩm [15]. Photo-SLAM đi xa hơn, giữ nguyên đặc trưng và hiệu chỉnh chùm tia cổ điển [16].

Giả định đã có một nguồn tư thế đủ tốt — và với bạn thì điều này đúng.

Giá phải trả bản đồ không còn đóng góp gì cho quỹ đạo; nó chỉ là bên tiêu thụ.

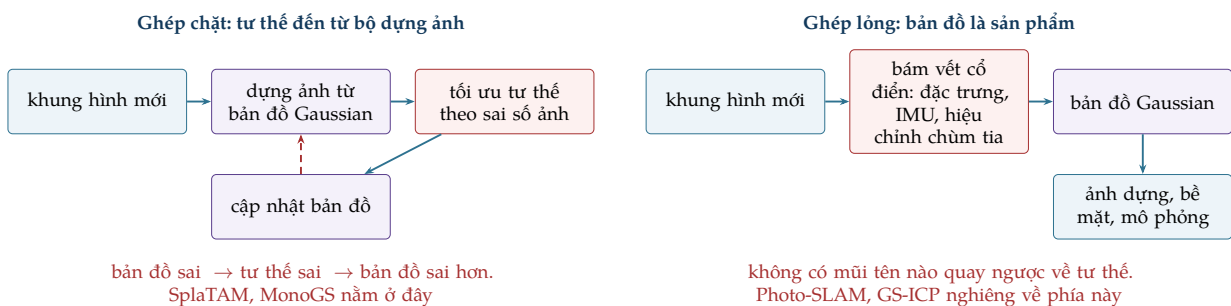
Hệ quả đây chính là mẫu ghép lỏng, và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm.
- 4 Vấn đề** Một triệu Gaussian không vừa ngân sách bộ nhớ và băng thông của máy nhúng.

Giải pháp	RTG-SLAM ép mỗi Gaussian hoặc đục hần hoặc trong hần, để một bề mặt cần ít hạt hơn nhiều [17].
Giả định	bề mặt trong cảnh phần lớn là đục, không bán trong suốt.
Giả phải trả	mất khả năng biểu diễn khói, kính, và vật phản chiếu.
Vấn đề mới	số hạt vẫn tăng theo diện tích đã đi qua, nên vấn đề chỉ được đẩy lùi chứ chưa được giải.

B. Sáu hệ, và điểm thiết kế đáng học của mỗi hệ

- **SplaTAM** — hệ RGB-D thuần Gaussian: bám vết bằng cách dựng ảnh từ bản đồ rồi tối ưu tư thế, dựng bản đồ bằng cách thêm Gaussian ở vùng chưa được phủ [18]. Điểm thiết kế đáng học là **mặt nạ hiển thị** (*visibility mask*): hệ chỉ tính mất mát ở các điểm ảnh mà bản đồ hiện có thật sự phủ tới. Nói theo ngôn ngữ của bạn, đó là một cổng loại bỏ phần dư không có cơ sở — cùng vai trò với việc bỏ các điểm không quan sát được trước khi tính phần dư trong hiệu chỉnh chùm tia.
- **MonoGS** — đưa 3DGS về đơn ảnh, và viết ra đạo hàm giải tích của sai số ảnh theo tư thế camera trong nhóm Lie [14]. Đây là bài trong nhóm gần với gu của một người làm tối ưu trên đa tạp nhất; nếu bạn định đọc mã nguồn một hệ 3DGS SLAM, hãy đọc hệ này.
- **GS-ICP SLAM** — tách hần bám vết khỏi dựng ảnh: tư thế do một biến thể ICP tổng quát tính trên chính tâm của các Gaussian, còn dựng ảnh chỉ là sản phẩm [15]. Đây là bằng chứng trực tiếp cho luận điểm trung tâm của chương: khi bỏ vòng lặp dựng ảnh ra khỏi đường ước lượng tư thế, hệ nhanh hơn hần, và chất lượng quỹ đạo không xấu đi.
- **Photo-SLAM** — giữ nguyên xương sống đặc trưng và hiệu chỉnh chùm tia cổ điển cho tư thế, gắn thêm một bản đồ Gaussian chỉ để dựng ảnh [16]. Đây là mẫu kiến trúc mà tôi cho là đáng bắt chước nhất cho một hệ đang chạy sản phẩm, vì nó không đặt tư thế vào tay bộ dựng ảnh.
- **RTG-SLAM** — tấn công thẳng vào số lượng Gaussian: ép mỗi Gaussian hoặc đục hần hoặc trong hần, để một bề mặt cần ít hạt hơn nhiều [17]. Nó cho thấy số Gaussian là một biến thiết kế chứ không phải hằng số của bài toán — điều rất quan trọng khi bạn có ngân sách bộ nhớ cố định.
- **OpenGS-SLAM** — dòng ghép ngữ nghĩa mở vào bản đồ Gaussian, để truy vấn bản đồ bằng ngôn ngữ. Tôi *không* chắc về chi tiết và kết quả của công trình này, nên chỉ nhắc tên và ý tưởng; phần ngữ nghĩa mở được bàn kỹ ở F/33.

Sáu hệ trên chia thành hai kiểu ghép nối, và phân biệt được hai kiểu đó quan trọng hơn nhớ tên từng hệ. Hình 1.5 vẽ cả hai.



Hình 1.5. Hai kiểu ghép nối, và đây là phân biệt quan trọng nhất của cả chương. Bên trái, tư thế được ước lượng bằng cách so ảnh dựng với ảnh thật, nên mọi lỗi bản đồ chây thẳng vào quỹ đạo; mũi tên đứt là vòng phản hồi đó. Bên phải, tư thế vẫn do đường cổ điển tính, còn bản đồ neural chỉ tiêu thụ tư thế; nó không thể làm quỹ đạo xấu đi, và cũng không thể làm quỹ đạo tốt lên. Chọn kiểu nào là quyết định kiến trúc, không phải chi tiết cài đặt.

Bảng 1.2. Dòng 3D Gaussian splatting làm bản đồ SLAM. Cột “tư thế đến từ đâu” quyết định hệ đó có thể làm hỏng quỹ đạo của bạn hay không.

Hệ	Tư thế đến từ đâu	Điểm thiết kế đáng học	Hỏng khi nào
SplaTAM	Dựng ảnh khác biệt được, RGB-D	Mặt nạ hiển thị: chỉ tính mất mát ở điểm ảnh bản đồ thật sự phủ	Độ sâu thiếu hoặc nhiều; chuyển động lớn giữa hai khung hình
MonoGS	Dựng ảnh, đơn ảnh, đạo hàm giải tích trên nhóm Lie	Viết thẳng Jacobian theo tư thế thay vì vi phân tự động	Tỉ lệ đơn ảnh; cảnh ít vân; chuyển động nhanh
GS-ICP SLAM	ICP tổng quát trên tâm Gaussian	Tách bám vết khỏi dựng ảnh, và hệ nhanh lên rõ rệt	Hình học nghèo nàn làm ICP trượt; vẫn cần độ sâu
Photo-SLAM	Đặc trưng + hiệu chỉnh chùm tia cổ điển	Bản đồ neural là bên tiêu thụ tư thế, không phải bên tạo ra nó	Bản đồ ảnh xấu ở nơi bám vết thừa khung hình chốt
RTG-SLAM	Bám vết dày, RGB-D	Ép Gaussian đục hoặc trong để giảm mạnh số hạt	Bề mặt bán trong suốt và vật phản chiếu
OpenGS-SLAM	<i>chưa kiểm chứng</i>	Ghép ngữ nghĩa mở vào bản đồ Gaussian	<i>chưa kiểm chứng</i>

C. Chỗ nổi rẻ nhất vào hệ bạn đang có

Nếu bạn muốn thử dòng này mà không đung vào đường ước lượng tư thế, có một chỗ nổi rất tự nhiên, và ít bài báo nói ra vì với họ nó hiển nhiên.

Bản 3DGS gốc khởi tạo các Gaussian từ đám mây điểm thừa của một pipeline dựng cảnh cổ điển. Hệ của bạn *đã* sinh ra đúng loại đám mây đó, mỗi khung hình chốt, kèm tư thế đã được hiệu chỉnh chùm tia. Nghĩa là bạn có sẵn đầu vào tốt nhất mà một hệ 3DGS có thể mơ tới: điểm 3D đã lọc ngoại lai, tư thế đã tối ưu toàn cục, và quan hệ đồng quan sát giữa các khung hình chốt.

Từ đó ra một kiến trúc ghép lỏng rất gọn, và nó không thêm rủi ro nào cho quỹ đạo:

- **Gieo hạt.** Mỗi điểm bản đồ thành một Gaussian, tỉ lệ khởi tạo theo khoảng cách tới các điểm lân cận.
- **Neo vào khung hình chốt.** Mỗi Gaussian ghi nhớ khung hình chốt đã sinh ra nó. Khi đóng vòng sửa tư thế khung hình đó, cả cụm Gaussian dịch theo bằng một phép biến đổi cứng.
- **Chia bản đồ con.** Dùng chính đồ thị đồng quan sát để cắt cảnh thành các bản đồ con, rồi chỉ tối ưu bản đồ con đang hoạt động. Đây là cách duy nhất giữ ngân sách bộ nhớ hữu hạn khi robot đi mãi.
- **Chạy lệch nhịp.** Khâu Gaussian chạy ở tần số thấp, trên một luồng riêng, và được phép rút khung hình. Nó là bên tiêu thụ, nên rút khung chỉ làm ảnh xấu đi chứ không làm hỏng quỹ đạo.

Đây không phải một phương pháp mới; nó là mẫu Photo-SLAM viết lại bằng các khối bạn đã có. Giá trị của nó nằm ở chỗ rủi ro gần bằng không: nếu bỏ hẵn khâu Gaussian đi, hệ quay về đúng trạng thái hiện tại, không mất gì.

D. Ba chỗ dòng này vẫn chưa chạm tới

Đọc xong bảng trên dễ có cảm giác vấn đề đã xong. Chưa. Ba chỗ vẫn để ngỏ, và cả ba đều nằm đúng chỗ hệ của bạn đau.

- **Chuyển động nhanh.** Bám vết bằng dựng ảnh là một bài toán tối ưu quang trắc, nên nó có bồn hút hữu hạn. Chuyển động lớn giữa hai khung hình đẩy nghiệm ra ngoài bồn, và cách

chứa duy nhất trong khuôn khổ đó là một dự đoán ban đầu tốt — tức là quay lại cần IMU hoặc cần ghép đặc trưng. Đây chính là bài học của BARF nói lại lần nữa.

- **Ảnh mờ do chuyển động.** Một khung hình mờ không phải quan sát nhiễu của cảnh tĩnh. Nó là tích phân của cảnh dọc một quãng chuyển động. Hàm mất mát quang trắc không có chỗ nào biểu diễn điều đó, nên tối ưu sẽ giải thích vết mờ bằng cách nhét thêm hình học sai. Các dòng khử mờ có ý thức về chuyển động, như BAD-NeRF và BAD-Gaussians, đưa mô hình phơi sáng vào bộ dựng ảnh; chúng được bàn ở F/34.
- **Phơi sáng và cân bằng trắng tự động.** Cùng một cơ chế: mô hình buộc phải giải thích thay đổi độ sáng bằng hình học. Các hệ trong chương này thường chạy trên chuỗi có phơi sáng cố định, và điều đó ít khi được ghi rõ.

E. Điều cả hai dòng đều thiếu: một bài toán tối ưu chung

Có một khoảng trống chung cho toàn bộ chương, và nó là khoảng trống mà một người quen đồ thị nhân tử sẽ thấy ngay.

Hiệu chỉnh chùm tia tối ưu tư thế và cấu trúc *cùng lúc*, trong một bài toán bình phương tối thiểu duy nhất. Nó cho ba thứ đi kèm: nghiệm hội tụ bậc hai gần cực tiểu, một **ma trận thông tin** (*information matrix*) để biết mình chắc chắn tới đâu, và một chỗ tự nhiên để cấm thêm nhân tử — tiền tích phân IMU, ràng buộc đóng vòng, tiên nghiệm hiệu chỉnh.

Gần như mọi hệ trong chương này làm khác hẳn. Chúng *luân phiên*: cố định bản đồ rồi tối ưu tư thế vài bước, cố định tư thế rồi tối ưu bản đồ vài bước. Đây là **hạ theo khối toạ độ** (*block coordinate descent*), và nó kém hơn ở đúng ba chỗ vừa kể.

- **Hội tụ chậm và phụ thuộc lịch trình.** Số bước mỗi phía trở thành siêu tham số, và nó tương tác với ràng buộc thời gian thực. Chạy trên máy chậm hơn nghĩa là chạy ít bước hơn, tức là một hệ *khác*. Đây là lý do sâu xa vì sao kết quả trên Orin không suy ra được từ kết quả trên máy để bàn.
- **Không có ma trận thông tin.** Không có hiệp phương sai của tư thế, nên không có gì để cấm vào một bộ lọc, không có gì để so bằng kiểm định chi bình phương, và không có cách nào phát hiện hệ đang không chắc chắn.
- **Không có chỗ cho IMU.** Đây là chỗ thiếu nghiêm trọng nhất với hệ của bạn. Trong khuôn khổ luân phiên, không có chỗ tự nhiên nào để đặt một nhân tử tiền tích phân IMU với trọng số đúng. Vài công trình gần đây có ghép IMU vào, thường bằng cách dùng nó làm dự đoán ban đầu; tôi *không* nắm chắc kết quả của chúng và không mô tả số liệu.

Nói ngắn gọn: dòng này mạnh về biểu diễn và yếu về ước lượng. Hệ của bạn thì ngược lại. Đó chính là lý do mẫu ghép lỏng không phải một thoả hiệp tạm bợ, mà là cách ghép đúng thể mạnh của hai bên.

I 1.4 Hai trục đánh giá, và vì sao chúng bị trộn lẫn

Đây là mục quan trọng nhất của chương. Nếu bạn chỉ đọc một mục, đọc mục này.

A. Hai câu hỏi hoàn toàn khác nhau

Một hệ trong chương này báo cáo hai nhóm con số. Nhóm thứ nhất là chất lượng ảnh dựng lại: PSNR, SSIM, LPIPS [19], [20]. Nhóm thứ hai là độ chính xác quỹ đạo: **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*), viết tắt ATE, đôi khi kèm RPE. Hai nhóm này trả lời hai câu hỏi khác nhau và có thể tách rời hoàn toàn.

Lý do tách rời đã được nêu ở C/16/03-nerf-3dgs-va-hop-nhat và đáng nhắc lại một câu: hàm mất mát chỉ ràng buộc ảnh dựng ra, không ràng buộc hình học. Màu phụ thuộc góc nhìn cho mô hình thêm một đường thoát: nó có thể “vẽ” thứ đáng lẽ phải “dựng”. Vì thế PSNR cao chứng minh mô hình khớp được các ảnh đã thấy, không chứng minh bản đồ đúng, và càng không chứng minh tư thế đúng.

Chiều ngược lại cũng đúng và ít được nói tới. Một hệ có ATE rất tốt vẫn có thể cho ảnh dựng xấu, đơn giản vì nó dành ít tài nguyên cho bản đồ. Photo-SLAM là ví dụ có chủ ý theo hướng ngược lại: quỹ đạo do đường cổ điển bảo đảm, còn ảnh đẹp là phần thêm.

Hình 1.6 bày cả hai trục lên một mặt phẳng. Đây là hình bạn nên vẽ lại cho chính hệ của mình.



Hình 1.6. Hai trục đánh giá trên một mặt phẳng; các nhãn là *góc phân tư khái niệm*, không phải vị trí đo được của hệ cụ thể nào. Bảng kết quả chỉ có cột PSNR không phân biệt được góc trên bên phải với góc dưới bên phải, và đó chính là chỗ người đọc bị dẫn sai. Quy tắc thực dụng: trước khi tin một hệ, hãy hỏi nó nằm ở đâu theo trục *đọc*, đo bằng đúng giao thức mà bạn dùng cho hệ hiện tại.

Bảng 1.3. Ba nhóm thước đo cho cùng một hệ. Cột cuối là cột hay bị bỏ qua nhất, và nó là lý do một hệ có thể đứng đầu bảng ở cột đầu mà vẫn không dùng được cho robot.

Nhóm	Đo cái gì	Trả lời được câu hỏi nào	Không chứng minh được gì
PSNR, SSIM, LPIPS	Ảnh dựng gần ảnh thật tới đâu	Bản đồ có dùng được để xem, để mô phỏng cảm biến không	Không nói gì về hình học, và không nói gì về tư thế
ATE, RPE	Quỹ đạo lệch chân trị bao nhiêu	Hệ có định vị được không — câu hỏi quyết định của SLAM	Không nói bản đồ có dày, có kín, có dùng để tránh vật cản được không
Sai số hình học: độ sâu L1, độ chính xác và độ phủ, F-score với lưới chân trị	Bề mặt dựng ra lệch bề mặt thật bao nhiêu	Bản đồ có dùng để lập kế hoạch, để đo đạc được không	Không nói ảnh có đẹp không, và cần chân trị hình học nên hiếm khi có ngoài đời

B. Ví dụ tính tay: vì sao một sai lệch hình học lớn chỉ tốn một điểm ảnh

Câu “PSNR không chứng minh hình học” nghe như một lời cảnh báo chung chung. Nó không phải vậy. Nó là một bất đẳng thức số học, và tính ra được trong ba dòng.

Trước khi tính, nói rõ một điều: **ví dụ dưới đây là một bài toán giả định hoàn toàn**. Mọi con số trong đó đều do mình đặt ra để thay vào công thức, không con số nào đến từ một phép đo trên thiết bị, cũng không con số nào trích từ một bài báo. Thứ đáng mang đi là *hình dạng* của quan hệ — Δu tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu — chứ không phải con số ở vế phải.

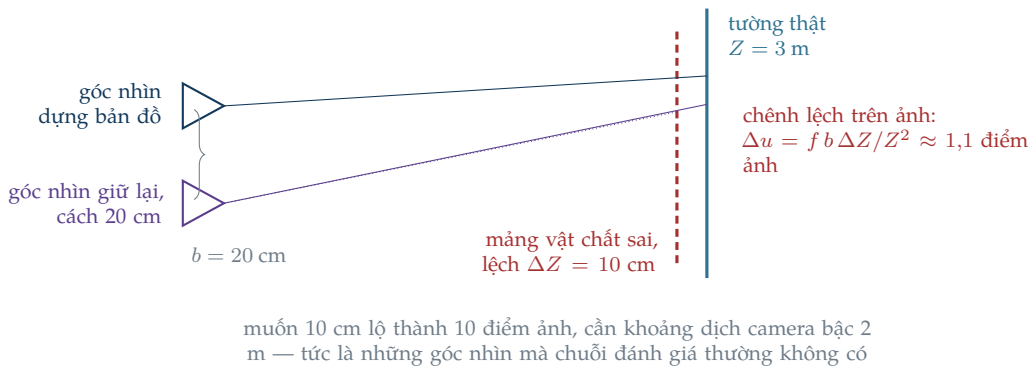
Xét một bức tường phẳng cách camera $Z = 3$ m. Giả sử bản đồ đặt sai một mảng vật chất, lệch $\Delta Z = 10$ cm về phía trước tường. Với robot, sai số này rất nghiêm trọng: nó là chênh lệch giữa dừng đúng và va vào tường. Bây giờ hỏi xem nó lộ ra bao nhiêu trong ảnh.

Một điểm ở độ sâu Z , nhìn từ một camera dịch ngang một đoạn b , có vị trí ảnh dịch đi một lượng tỉ lệ với fb/Z . Sai một lượng ΔZ làm vị trí ảnh lệch đi

$$\Delta u \approx \frac{fb\Delta Z}{Z^2} = \frac{500 \times 0,2 \times 0,1}{3^2} \approx 1,1 \text{ điểm ảnh},$$

với tiêu cự $f = 500$ điểm ảnh và khoảng dịch camera $b = 20$ cm. *Một điểm ảnh*. Trên một ảnh mà mô hình còn được phép chỉnh màu theo góc nhìn, một điểm ảnh chìm hoàn toàn trong nhiễu cảm biến và trong sai số nội suy. PSNR gần như không đổi. Bản đồ thì sai 10 cm.

Hình 1.7 vẽ đúng hình học của phép tính đó.



Hình 1.7. Vì sao sai số hình học trốn được khỏi PSNR; các con số trên hình là *giả định* của ví dụ, không phải số đo. Với những giá trị giả định đó, sai 10 cm ở tầm 3 m chỉ dời điểm ảnh khoảng một điểm ảnh khi góc nhìn kiểm tra chỉ cách góc nhìn dựng bản đồ 20 cm. Độ nhạy tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu. Hệ quả thực hành: chỉ khi tập góc nhìn giữ lại *cách xa* quỹ đạo chụp thì chất lượng ảnh mới bắt đầu nói được điều gì về hình học — và đó đúng là loại tập đánh giá mà các bài trong chương này hiếm khi dựng.

Ba hệ quả rút ra ngay, và cả ba đều dùng được khi bạn đọc bảng của người khác.

- **Độ nhạy tỉ lệ nghịch với bình phương độ sâu.** Vật xa hoàn toàn không bị ràng buộc bởi ảnh. Đây là lý do các bản đồ neural dựng ngoài trời hay có “sương mù” vật chất giả ở nền xa.
- **Độ nhạy tỉ lệ thuận với khoảng dịch camera.** Một tập góc nhìn giữ lại nằm sát quỹ đạo chụp gần như không kiểm tra được hình học. Muốn kiểm tra thật, phải giữ lại những góc nhìn *lệch hẳn* khỏi quỹ đạo.
- **Muốn ràng buộc hình học thì phải đo hình học.** Không có cách nào lách: nếu đầu ra sẽ được dùng để tránh vật cản, thước đo phải là sai số bề mặt so với một quét chân trị, không phải PSNR.

C. Bảy câu hỏi phải hỏi trước khi tin một bảng kết quả

Phần lớn kết quả trong chương này được công bố trong một điều kiện rất hẹp, và điều kiện đó thường nằm rải rác trong phần thí nghiệm chứ không nằm ở bảng. Bảy câu hỏi dưới đây lọc được gần hết.

1. **Ảnh được đo trên góc nhìn nào?** Rất nhiều bảng báo cáo PSNR trên chính các khung hình đầu vào hoặc các khung hình chốt. Con số đó đo *độ khớp*, không đo khả năng tổng quát sang góc nhìn mới. Hai đại lượng này khác nhau rất xa với một mô hình có hàng triệu tham số cho một cảnh.
2. **Tư thế lúc dựng bản đồ là tư thế ước lượng hay chân trị?** Một số thí nghiệm chạy ở chế độ “chỉ dựng bản đồ” với tư thế chân trị. Kết quả đó hợp lệ, nhưng nó nói về bản đồ chứ không nói về hệ SLAM.
3. **Độ sâu đến từ đâu?** Replica cho độ sâu tổng hợp hoàn hảo [21]. Cảm biến thật cho độ sâu có nhiễu, có lỗ, và có giới hạn tầm. Một hệ dựa nặng vào độ sâu sẽ tụt hạng rất nhiều giữa hai điều kiện này.
4. **Chuỗi dài bao nhiêu và có vòng không?** Phần lớn chuỗi trong Replica và TUM RGB-D chỉ dài vài chục giây tới vài phút, đi bộ trong một phòng [22], [23]. Trôi tích lũy và đóng vòng gần như không được kiểm tra.
5. **ATE được căn chỉnh thế nào?** Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh có cả tỉ lệ. Phép căn chỉnh đó *hấp thụ* sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất của hệ đơn ảnh. Nếu bạn so với hệ stereo-inertial của mình, hai con số không cùng đơn vị nghĩa.
6. **Phần cứng nào và tốc độ nào?** Con số fps trong bài gần như luôn là GPU để bàn. Và “thời gian thực” trong nhiều bài nghĩa là xử lý kịp tốc độ phát lại đã giảm, không phải kịp tốc độ cảm biến.
7. **Baseline cổ điển có được tinh chỉnh tử tế không?** Đây là câu hỏi của E/24 áp vào đây. Một bảng so bản đồ neural với ORB-SLAM3 chạy cấu hình mặc định trên chuỗi mà cấu hình đó không hợp là một so sánh không có nghĩa.

CẠM BÃY

Cái bẫy hay gặp nhất không phải số liệu sai mà là *ghép hai kết luận đúng thành một kết luận sai*. Bài báo nói “PSNR của chúng tôi cao nhất bảng” — đúng. Bài báo nói “ATE của chúng tôi tốt hơn NICE-SLAM” — cũng đúng. Người đọc ghép lại thành “hệ này tốt hơn hệ tôi đang chạy” — sai, vì cả hai hệ được so đều thuộc dòng neural, và cả hai đều có thể tệ hơn một hệ đặc trưng cổ điển đã tinh chỉnh trên cùng chuỗi. Cách kiểm tra rẻ nhất là tìm trong bài xem có dòng nào so với một hệ cổ điển *trên cùng giao thức căn chỉnh* hay không. Nếu không có, bạn chưa biết gì về vị trí của nó theo trục dọc trong Hình 1.6.

Bảng 1.4. Điều kiện ẩn sau bốn tập dữ liệu hay gặp. Bảng này giải thích vì sao thứ hạng trên hai dòng đầu gần như không dự đoán được hành vi trên dòng cuối — tập dữ liệu của chính bạn.

Tập	Độ sâu	Độ dài, vòng	Điều kiện để bị bỏ qua
Replica	Tổng hợp, hoàn hảo, không lỗ	Ngắn, quỹ đạo dựng sẵn	Không nhiều ảnh, không mờ, phơi sáng cố định; đây là điều kiện dễ nhất có thể
TUM RGB-D	Cảm biến kết cấu ánh sáng, có nhiều và lỗ	Vài chục giây, ít vòng lớn	Trong nhà, đi bộ, một phòng; chuyển động vẫn khá êm
ScanNet	Cảm biến, quét tay	Dài hơn, có quay lại chỗ cũ	Ảnh mờ và phơi sáng tự động xuất hiện thật; chân trị tư thế cũng từ một hệ dựng lại
EuRoC, chuỗi của bạn	Stereo, không có độ sâu trực tiếp	Vài phút, chuyển động mạnh	Bay trong không gian rộng, chuyển động nhanh, ảnh mờ, thay đổi sáng — gần như mọi giả định của chương đều bị vi phạm

D. Ba thí nghiệm bạn phải tự chạy

Đọc bảng của người khác chỉ để loại bớt ứng viên. Muốn biết một hệ có giúp được mình không, có ba thí nghiệm, và chúng đi theo đúng thứ tự kinh tế: rẻ trước, đắt sau.

- **Thí nghiệm oracle tư thế.** Cấp tư thế chân trị cho hệ và chỉ để nó dựng bản đồ. Con số thu được là *trần* chất lượng bản đồ khi khâu tư thế hoàn hảo. Nếu trần đó đã không đủ cho việc hạ nguồn của bạn, mọi công sức về sau đều bị chặn trên. Đây đúng là công cụ ở E/24, áp vào đây.
- **Thí nghiệm tách ghép nối.** Chạy cùng hệ ở hai chế độ: tư thế do bộ dựng ảnh tính, và tư thế do đường cổ điển của bạn tính. Hiệu số ATE giữa hai chế độ trả lời thẳng câu hỏi “bản đồ neural có làm quỹ đạo tốt lên không”. Kinh nghiệm đọc các bảng đã công bố cho thấy câu trả lời thường là không; nhưng đó là kết luận rút từ bảng của người khác, không phải phép đo của tôi.
- **Thí nghiệm ngoài phân phối.** Chạy trên chính chuỗi khó của bạn: chuyển động nhanh, ảnh mờ, phơi sáng tự động, ngoài trời. Đây là chỗ khoảng cách giữa bảng công bố và thực tế lộ ra nhiều nhất, và cũng là chỗ rẻ nhất để phát hiện sớm.

Và một nhắc nhở về kỷ luật đo, vì nó đã cản bạn ở chỗ khác: mọi con số trên đây phải chạy nhiều lần. Các hệ này có thêm hai nguồn tản mát mà hệ cổ điển không có — khởi tạo ngẫu nhiên của bản đồ và số bước tối ưu thực sự chạy được dưới ràng buộc thời gian thực. Sàn nhiễu vì thế thường rộng hơn bạn quen.

E. Dựng tập góc nhìn giữ lại cho đúng

Hai mục trên gộp lại thành một yêu cầu rất cụ thể, và nó đáng viết riêng vì hầu như không ai làm đúng.

Cách làm mặc định là giữ lại mỗi khung hình thứ tám của chính chuỗi đầu vào. Cách này rẻ và gần như vô dụng. Lý do đã tính ở mục B: khung hình thứ tám chỉ cách khung hình lân cận vài centimét, mà $\Delta u \approx fb\Delta Z/Z^2$ tỉ lệ thuận với khoảng dịch b . Thu b từ hàng chục centimét xuống vài centimét thì độ nhạy tụt theo đúng tỉ lệ ấy, và với các giá trị giả định của mục B thì sai lệch hình học lộ ra ở mức nhỏ hơn một phần mười điểm ảnh — tức là chìm hẳn dưới nhiễu. Bạn đang đo khả năng nội suy giữa hai ảnh liền kề, không đo bản đồ.

Ba cách dựng tập giữ lại có nghĩa, xếp theo công sức tăng dần:

- **Giữ lại theo đoạn, không theo khung hình.** Cắt bỏ nguyên một đoạn quỹ đạo dài vài giây khỏi dữ liệu dựng bản đồ, rồi đánh giá trên đoạn đó. Khoảng dịch camera khi ấy tính bằng mét chứ không bằng centimét.
- **Quay hai lượt.** Đi qua cùng một chỗ hai lần theo hai đường khác nhau, dựng bản đồ bằng lượt một, đánh giá bằng lượt hai. Đây là cách rẻ nhất cho dữ liệu tự thu, và nó cũng cho bạn luôn một phép thử đóng vòng.
- **Đo thẳng hình học.** Nếu có một quét chân trị — máy quét laser, hoặc một mô hình đã dựng kỹ ngoại tuyến — thì bỏ hẳn ảnh đi và đo sai số bề mặt. Đây là câu trả lời đúng khi đầu ra sẽ được dùng để tránh vật cản.

Nếu bạn chỉ đọc bảng của người khác, câu hỏi rút gọn thành một dòng: tập giữ lại của họ cách quỹ đạo chụp bao xa? Bài nào không trả lời được câu đó thì cột PSNR của nó chỉ nói về nội suy.

1.5 Ngân sách trên Jetson Orin, và câu hỏi có đáng cho robot chưa

Mọi thứ ở trên đều giả định có một GPU để bàn. Mục này hỏi chi phí ấy có *hình dạng* gì trên phần cứng bạn thật sự triển khai. Mục này cố tình **không đưa ra con số nào**: mình không có phép đo trên Orin cho bất kỳ hệ nào trong chương, và một ước lượng tự tính trên giấy rất dễ bị đọc lại thành một số đo. Cái đáng học ở đây không phải độ lớn, mà là *thừa số nào nhân với thừa số nào* — vì chính cấu trúc đó nói cho bạn biết nên cắt ở chỗ nào, và nó đúng bất kể con số cụ thể là bao nhiêu.

A. Bộ nhớ: đếm bằng công thức, không quy ra con số

Một Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số màu bậc ba — gần sáu chục số thực cho *một* hạt. Đã là bội số lớn so với một điểm bản đồ thưa, vốn chỉ cần ba số tọa độ cộng một bộ mô tả ngắn. Và một cảnh cỡ căn phòng dựng cho tử tế cần bậc một triệu hạt như thế.

Nhưng bộ nhớ tham số chưa phải khoản phải trả. Trong SLAM, bản đồ *đang được tối ưu* chứ không nằm yên. Mỗi tham số cần thêm một ô gradient và hai ô trạng thái của bộ tối ưu kiểu Adam, nên bộ nhớ thực tế là bốn lần bộ nhớ tham số:

$$\text{bộ nhớ bản đồ} \propto \underbrace{4}_{\text{tham số, gradient, hai ô Adam}} \times \underbrace{59}_{\text{số thực mỗi hạt}} \times \underbrace{N}_{\text{số Gaussian}}.$$

Thừa số duy nhất bạn điều khiển được là N ; hai thừa số kia là hằng số của biểu diễn. Đó là lý do mọi công trình muốn đưa 3DGS lên máy nhúng đều tấn công vào N chứ không vào chỗ nào khác.

Hướng của kết luận thì rõ, và nó không cần một con số nào: **bản đồ Gaussian tốn bộ nhớ hơn đám mây điểm thưa vài bậc độ lớn cho cùng một căn phòng, và bộ nhớ mới là ràng buộc quyết định của cả dòng này chứ không phải tốc độ dựng ảnh**. Muốn biết bao nhiêu megabyte trên cảnh của bạn thì phải đo trên cảnh của bạn: số hạt mà một cảnh cần phụ thuộc vào chính cảnh đó, nên không có hằng số nào chuyển thẳng được từ bài báo sang.

Dung lượng nghe thì vẫn có vẻ vừa một Orin. Nhưng bộ nhớ trên Orin là bộ nhớ *dùng chung* giữa CPU và GPU, và nó đang phải chứa cả hệ SLAM, cả các mạng nhận thức khác, cả ROS. Quan trọng hơn, dung lượng không phải ràng buộc cản trước.

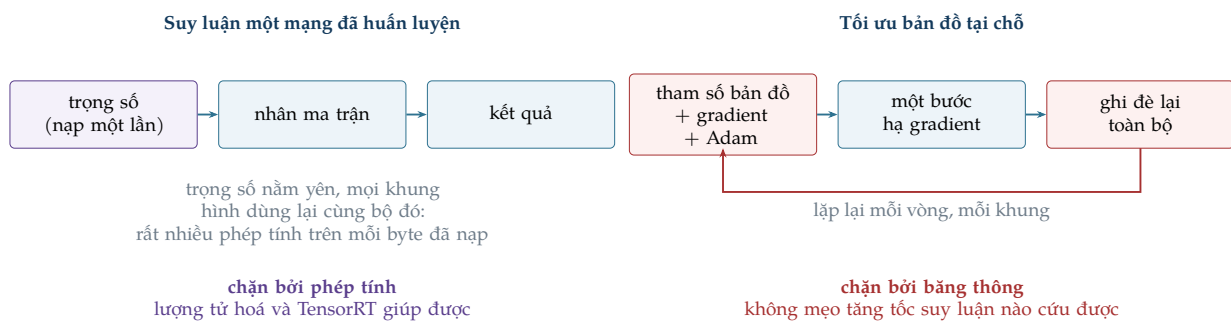
B. Bảng thông mới là ràng buộc cần trước

Mỗi vòng tối ưu phải đọc *toàn bộ* tham số cùng gradient và trạng thái bộ tối ưu, rồi ghi lại chúng. Viết lưu lượng ấy thành công thức là đủ thấy vấn đề:

$$\text{lưu lượng mỗi giây} \propto \underbrace{N}_{\text{số hạt}} \times \underbrace{4}_{\text{tham số+gradient+Adam}} \times \underbrace{2}_{\text{đọc rồi ghi}} \times \underbrace{n_{\text{vòng}}}_{\text{vòng tối ưu mỗi khung}} \times \underbrace{f}_{\text{khung mỗi giây}}.$$

Năm thừa số, và bốn trong năm là hằng số của kiến trúc chứ không phải của cài đặt. Kết luận định tính rút ra: với một bản đồ cỡ căn phòng, riêng việc *di chuyển* tham số qua lại đã ăn phần lớn băng thông bộ nhớ của một máy nhúng, trước khi thực hiện một phép nhân nào. Đây là lý do cơ học — không phải lý do kỹ thuật cài đặt — khiến một hệ 3DGS SLAM đầy đủ chưa chạy được ở tốc độ cảm biến trên phần cứng nhúng. Mình không có phép đo để nói nó hút bao nhiêu lần; công thức chỉ nói nó hút theo hướng nào, và cắt thừa số nào thì đỡ. Ba hướng thoát đều tồn tại và đều phải trả giá: giảm số Gaussian như RTG-SLAM, giảm số vòng tối ưu mỗi khung, hoặc chạy khâu bản đồ ở tần số thấp hơn hẳn khâu bám vết.

Hình 1.8 nói vì sao khoản này khác hẳn mọi khoản học sâu khác bạn từng đưa lên Orin.



Hình 1.8. Vì sao khoản chi phí ở chương này không giống bất kỳ khoản học sâu nào khác trong Phần F. Bên trái là thứ bạn đã quen triển khai: trọng số nạp một lần rồi nằm yên, nên mỗi byte đã nạp được dùng cho rất nhiều phép tính và nút thắt nằm ở phép tính. Bên phải là thứ đang chạy trong một hệ SLAM neural: chính tham số bản đồ là biến đang bị sửa, nên mỗi vòng phải đọc rồi ghi lại toàn bộ, và số phép tính trên mỗi byte rất thấp. Mũi tên đỏ quay ngược là toàn bộ khác biệt — nó biến bài toán từ “chặn bởi phép tính” thành “chặn bởi băng thông”, và đó là lý do lượng tử hoá cùng TensorRT gần như không giúp gì.

Còn một ràng buộc nữa mà bảng thông số không nói: nhiệt và điện. Orin chạy ở nhiều mức công suất, và mức thấp cắt cả xung nhịp GPU lẫn băng thông. Một khối chiếm gần trọn băng thông trong nhiều phút sẽ kéo cả hệ vào vùng giảm xung, nên các khối khác — phát hiện vật thể, lập kế hoạch — chậm theo. Đây là dạng ghép nối không xuất hiện trong bất kỳ phép đo đơn lẻ nào, và chỉ lộ ra khi chạy cả hệ trong thời gian dài.

Với dòng NeRF, chỗ nghẽn dời từ băng thông sang phép tính, và công thức đổi theo. Chi phí một vòng tỉ lệ với (số tia lấy mẫu) $\times$ (số mẫu đọc mỗi tia) $\times$ (chi phí một lần chạy MLP), rồi vẫn phải nhân tiếp với số vòng mỗi khung và số khung mỗi giây. Ba thừa số đầu đều là hằng số kiến trúc, và tích của chúng lớn vì mỗi tia phải dò chứ không *chiếu* — đúng cơ chế đã nói ở mục 3. Hướng kết luận giống hệt dòng Gaussian: một khối duy nhất ăn phần lớn ngân sách của máy, trong khi máy còn phải chạy cả phần nhận thức lẫn phần điều khiển. Lần nữa, đây là hình dạng của bài toán chứ không phải một phép đo; con số thật chỉ có nếu bạn chạy trên chính thiết bị của mình.

Một phản xạ không dùng được ở đây. Khi thấy một mô hình quá chậm trên Orin, phản xạ quen thuộc là lượng tử hoá và biên dịch bằng TensorRT. Ở đây phản xạ đó gần như không giúp gì, và lý do đáng nhớ: cái đang chạy không phải *suy luận* mà là một *vòng lặp tối ưu* trên chính tham số bản đồ. Lượng tử hoá làm hỏng gradient; đồ thị tính toán thay đổi liên tục vì Gaussian được thêm và bị cắt; và phần lớn thời gian nằm ở lưu lượng bộ nhớ chứ không ở phép nhân. Đây là chỗ

khác biệt căn bản giữa chương này và mọi chương khác của Phần F, nơi ta triển khai một mạng đã huấn luyện xong.

C. Vì sao gần như mọi kết quả đều là trong nhà và chuỗi ngắn

Ba ràng buộc trên giải thích luôn một điều mà người đọc thường coi là ngẫu nhiên: vì sao gần như toàn bộ kết quả trong chương này đến từ cảnh trong nhà, chuỗi vài chục giây tới vài phút, có độ sâu RGB-D, chạy trên GPU để bàn.

- **Trong nhà** vì cảnh có biên, nên lưới hoặc tập Gaussian cấp phát được. Ngoài trời không có biên, và bộ nhớ tăng theo diện tích đã đi qua.
- **Chuỗi ngắn** vì bản đồ chỉ lớn thêm chứ không bao giờ nhỏ đi, và vì chuỗi ngắn thì trôi tích lũy chưa kịp thành vấn đề.
- **RGB-D** vì độ sâu cho hình học gần như miễn phí. Bỏ độ sâu đi thì phải mượn tiên nghiệm học được, và kéo theo mọi giới hạn của tiên nghiệm đó.
- **GPU để bàn** vì ở đó cả dung lượng lẫn băng thông bộ nhớ đều dư ra rất nhiều so với một máy nhúng, và lại không phải chia cho phần còn lại của robot.
- **Camera đi chậm và đều** vì bám vết quang trắc có bồn hút hữu hạn, và vì ảnh mờ phá thẳng vào hàm mất mát.

Không điều nào ở trên là lỗi của các tác giả. Đó là những điều kiện cần để bài toán giải được ở trạng thái hiện tại của lĩnh vực. Nhưng chúng *là* khoảng cách giữa bảng kết quả và chiếc robot của bạn, và đó là khoảng cách bạn phải tự trả.

D. Điền số vào bốn cổng

Khung bốn cổng ở F/27 có ích nhất khi nó được điền số thật thay vì cảm giác. Bảng 1.5 điền cho chương này, với ba nhu cầu hạ nguồn khác nhau. Chú ý là cùng một công nghệ cho ba kết luận khác nhau, và khác biệt nằm ở nhu cầu chứ không ở công nghệ.

Bảng 1.5. Bốn cổng của F/27 điền cho bản đồ neural, tách theo nhu cầu hạ nguồn. Cùng một công nghệ, ba kết luận khác nhau — và đó là lý do câu hỏi “bản đồ neural có đáng không” không có câu trả lời chung.

Cổng	Để định vị	Để tránh vật cản	Để dựng lại và xem
Bài toán có thật không	Không: hệ hiện tại đã định vị tốt	Có, nhưng TSDF và lưới chiếm dụng đã giải	Có, và không có lời giải cổ điển nào thay được
Đo được không	Có: ATE, cùng giao thức căn chỉnh	Có, nhưng cần chân trị hình học, thứ khó có ngoài đời	Có: PSNR, SSIM, LPIPS trên góc nhìn giữ lại thật sự
Vừa ngân sách Orin không	Không: riêng lưu lượng đọc-ghi tham số mỗi vòng đã ăn phần lớn băng thông	Không ở tốc độ cảm biến; có nếu chạy khâu bản đồ ở tần số thấp	Có, nếu chạy ngoại tuyến trên máy trạm
Hồng ra sao	Im lặng, qua vòng phản hồi dương, không có chỉ báo	Vật chất giả lơ lửng bị coi là vật cản, hoặc ngược lại	Ảnh xấu đi, thấy ngay bằng mắt — chế độ hồng lạnh nhất
Kết luận	Chưa đáng	Dùng TSDF trước	Đáng, và nên làm ngoại tuyến

Hàng cuối là chỗ đáng nhớ. Cùng một đồng mã nguồn, cùng một bài báo, mà kết luận đảo hẳn theo việc bạn định dùng đầu ra vào việc gì. Ai hỏi “bản đồ neural có đáng không” mà không nói rõ dùng để làm gì thì đang hỏi một câu không có đáp án.

E. Kết luận thẳng thắn: có đáng chưa

Trả lời tách theo mục đích, vì gộp chung là chỗ mọi tranh luận đi vào ngõ cụt.

- **Thay bản đồ định vị bằng bản đồ neural: chưa đáng.** Không có bằng chứng nào cho thấy nó cải thiện ATE của một hệ stereo-inertial đã tinh chỉnh kỹ, trên chuỗi thật, ở ngân sách của Orin. Đổi lại bạn mất tính chắc chắn đoán được, mất tính lặp lại, và tiêu gần hết ngân sách tính toán. Đây là kết luận đọc từ điều kiện thí nghiệm của các bài đã công bố, không phải từ một phép đo của tôi — nhưng nó là kết luận mà chính các điều kiện đó bắt buộc.
- **Bản đồ neural như một sản phẩm phụ, chạy ở tần số thấp: đáng thử, nếu có nhu cầu thật.** Nếu bài toán hạ nguồn cần ảnh — xem lại từ xa, mô phỏng cảm biến, sinh dữ liệu huấn luyện, kiểm tra công trình — thì mẫu ghép lỏng của Photo-SLAM là cách vào rẻ và không rủi ro. Tư thế vẫn do đường cổ điển bảo đảm, nên xấu nhất là bạn mất một luồng GPU.
- **Bản đồ dày để tránh vật cản: chọn TSDF hoặc lưới chiếm dụng trước.** Chúng rẻ hơn nhiều bậc, có mô hình xác suất, có bề mặt định nghĩa sạch, và đã được kiểm chứng lâu năm. Chỉ chuyển sang biểu diễn neural khi bạn chỉ ra được một việc cụ thể mà chúng không làm nổi.
- **Dựng lại chất lượng cao ngoại tuyến: đáng, và đây là chỗ dòng này thật sự thắng.** Bỏ ràng buộc thời gian thực đi thì mọi phép tính ở trên đối hân. Ghi bag trên robot, chạy dựng bản đồ trên máy trạm, đó là cách dùng đúng sức của công nghệ này hôm nay.

F. Bốn bài học thiết kế đáng giữ kể cả khi không dùng

Đây là phần trả lời câu hỏi thứ tư của khung đọc: nếu không lấy nguyên phương pháp thì lấy được gì.

- **Chọn biểu diễn theo tiêu chí “sửa được sau khi đóng vòng”.** Đây là tiêu chí phân loại mạnh nhất trong cả chương, và nó tách dòng Gaussian khỏi dòng MLP đơn rõ hơn mọi bảng PSNR. Nó cũng đúng cho mọi thứ khác bạn gắn vào bản đồ: bản đồ ngữ nghĩa, bản đồ chiếm dụng, bộ nhớ đóng vòng.
- **Mọi hàm mất mát quang trắc cần một lịch làm trơn.** Bài học của BARF, và nó là bản sao của kim tự tháp ảnh trong các hệ trực tiếp cổ điển. Nếu bạn thêm một số hạng quang trắc vào hệ mình mà không có lịch làm trơn, nó sẽ mắc kẹt và bạn sẽ đổ lỗi nhầm cho khởi tạo.
- **Chốt khung hình theo lượng thông tin mới, đo được.** iMAP đo bằng sai số dựng ảnh dưới bản đồ hiện tại. Ý tưởng đó chuyển được sang hệ cổ điển: thay ngưỡng “số điểm khớp dưới 90%” bằng một đại lượng có nghĩa thống kê. Bạn đã biết chính sách chốt khung hình là chỗ nhạy cảm bậc nhất của hệ mình.
- **Trọng số của phần dư phải đến từ bất định, không đến từ hằng số.** NeRF-SLAM cân hàm mất mát dựng bản đồ bằng bất định của độ sâu. Đây đúng là tư duy ma trận thông tin, và nó là sợi dây nối chương này với F/34.

CẦN NHỚ

Hai câu hỏi phải luôn tách rời: *bản đồ này trông có đẹp không* và *quỹ đạo này có đúng không*. Chúng được tối ưu bởi hai hàm mất mát khác nhau, đo bằng hai bộ thước đo khác nhau, và một hệ hoàn toàn có thể thắng ở câu thứ nhất trong khi thua ở câu thứ hai. Quy tắc kiến trúc rút ra: đừng để bộ dựng ảnh là nguồn của tư thế, trừ khi bạn đã tự đo và thấy nó tốt hơn đường cổ điển của mình trên chính chuỗi khó của mình. [T3]

1.6 Giới hạn và trường hợp hòng

Giới hạn nặng nhất đã nói ở trên và đáng nhắc lại theo cách khác: các hệ này tối ưu *riêng cho từng cảnh*. Chi phí vì thế là chi phí lặp lại, không phải chi phí một lần như huấn luyện một mạng rồi triển khai. Mọi kỹ thuật tăng tốc suy luận mà bạn quen đều không áp được. Các dòng mô hình tiến thẳng, dự đoán thẳng tập Gaussian từ vài ảnh, đang xoá dần ranh giới này; tại thời điểm viết, chất lượng của chúng vẫn dưới bản tối ưu riêng từng cảnh, và đây là quan sát xu hướng chứ chưa phải kết quả đã ổn định.

Chế độ hỏng thứ hai là hỏng *im lặng* và nó đáng sợ nhất. Khi tư thế sai vài phần mười độ, tối ưu không báo lỗi. Nó bù bằng cách sinh ra vật chất giả lơ lửng, và ảnh dựng ra vẫn đẹp. Với một hệ ghép chặt, vật chất giả đó lại quay lại làm sai tư thế của khung hình sau. Bạn có một vòng phản hồi dương mà không có chỉ báo nào bật lên. Ở hệ cổ điển, cùng tình huống ấy làm số điểm khớp tụt và làm phần dư tăng — tức là bạn *nhìn thấy* nó. Mất tính chẩn đoán được là khoản mất ít được nhắc nhất và đắt nhất khi triển khai.

Chế độ hỏng thứ ba nằm ở giả định về quá trình tạo ảnh. Cảnh phải tĩnh, phơi sáng phải nhất quán, ảnh không được mờ. Ba giả định này bị vi phạm liên tục trên một robot thật, và chúng bị vi phạm *cùng lúc* đúng vào những đoạn khó: robot quay nhanh trong hành lang thiếu sáng, camera tự tăng thời gian phơi, ảnh mờ đi, người đi qua khung hình. Mỗi vi phạm đều được mô hình giải thích bằng cách thêm hình học sai, vì đó là biến duy nhất nó có.

Cuối cùng, một giới hạn về kiến thức chứ không về công nghệ: tôi không có phép đo riêng nào cho các hệ trong chương này trên phần cứng Orin, và cũng không kiểm chứng được các con số hiệu năng mà các bài báo đưa ra. Vì thế mục 1.5 cổ tình chỉ nói về *cấu trúc* của chi phí — thừa số nào nhân với thừa số nào — chứ không đưa ra con số nào. Cấu trúc đó đủ để nói “khoản này bị chặn bởi băng thông chứ không bởi phép tính” và “thừa số duy nhất cắt được là số hạt”. Nó không đủ để nói hệ nào nhanh hơn hệ nào, cũng không đủ để nói hệt bao nhiêu lần. Nếu bạn cần một con số, chỉ có một cách lấy nó, là chạy trên chính thiết bị của mình.

1.7 Thực hành

Repo và tài nguyên. Mã nguồn của các hệ trong chương đều mở: [cvg/nice-slam](#), [HengyiWang/Co-SLAM](#), [idiap/ESLAM](#), [eriksandstroem/Point-SLAM](#), [youmi-zym/GO-SLAM](#) cho dòng NeRF; [spla-tam/SplaTAM](#), [muskie82/MonoGS](#), [Lab-of-AI-and-Robotics/GS_ICP_SLAM](#), [HuajianUP/Photo-SLAM](#) cho dòng Gaussian. Bản 3DGS gốc là [graphdeco-inria/gaussian-splatting](#). Để đo, dùng lại chính *evo* mà bạn đã quen cho ATE, và một script nhỏ tính PSNR cùng LPIPS trên tập góc nhìn giữ lại. Tập dữ liệu vào cửa nhanh nhất là TUM RGB-D vì nó nhỏ và mọi hệ đều báo cáo trên đó. Danh sách tên hệ ở đây chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các hạ tầng bền hơn nhiều là bộ rasterize gốc, *evo*, và *opencv*.

Câu hỏi kiểm tra hiểu. Vì sao lưới đặc trưng chứa được quên thăm khốc trong khi tăng số tham số của một MLP đơn thì không? Một bài báo cáo PSNR đứng đầu bảng trên Replica và ATE nhỏ hơn mọi hệ neural khác được so: hai kết quả đó nói gì và không nói gì về việc hệ chạy được trên chuỗi EuRoC của bạn? Nếu chỉ được chạy một thí nghiệm để quyết định có theo đuổi hướng này không, bạn chạy thí nghiệm nào, và nó chặn trên cái gì?

Mini project. Xem khối *Mini project SLAM* bên dưới; nó chính là thí nghiệm rẻ nhất trả lời được câu hỏi trung tâm của chương.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bản đồ điểm thưa	Vài chục tới vài trăm nghìn điểm 3D kèm bộ mô tả. Đủ để trả lời “tôi ở đâu”, không trả lời được “chỗ kia có trống không” vì nó chưa bao giờ mô hình hoá khoảng không giữa các điểm.
••• TSDF	Lưới lưu khoảng cách có dấu tới bề mặt gần nhất. Bề mặt là tập mức 0 nên có định nghĩa sạch, và đây là đối thủ thật sự của bản đồ neural chứ không phải bản đồ thưa.
•• iMAP	Hệ SLAM RGB-D dùng một MLP làm toàn bộ bản đồ. Gọn tới mức một phòng chỉ cần vài trăm nghìn tham số, nhưng trọng số dùng chung nên học chỗ mới làm mờ chỗ cũ.
••• Quên thảm hoạ	Học dữ liệu mới làm mất khả năng tái tạo dữ liệu cũ, vì cùng bộ tham số gánh cả hai. Triệu chứng trong bản đồ neural: sang phòng bên rồi quay lại thì phòng cũ mờ đi.
••• Vì sao lưới đặc trưng chưa được quên thảm khốc?	Vì nó thu hẹp giá đỡ của gradient. Một tia chỉ chạm vài ô lưới quanh nó thay vì chạm mọi trọng số, nên cập nhật trở thành cục bộ. Cái giá là bộ nhớ quay lại phụ thuộc thể tích cảnh.
•• Ba cách cấp phát thưa	Co-SLAM ghép mã hoá toạ độ với bảng băm; ESLAM dùng ba mặt phẳng đặc trưng nên bộ nhớ theo bình phương chứ không theo lũy thừa ba; Point-SLAM neo đặc trưng lên chính điểm đã quan sát.
••• BARF	Tối ưu tư thế cùng lúc với trường bức xạ. Phát hiện chính: mã hoá vị trí tần số cao làm mặt phẳng mất mát lờ mờ, nên phải bật tần số từ thô tới tinh.
••• Vì sao tần số cao phá gradient theo tư thế?	Vì đạo hàm của thành phần tần số k có biên độ tỉ lệ với k . Các gọn tần số cao sinh ra vô số cực tiểu địa phương gần nhau, và hạ gradient rơi vào cái hố gần nhất.
•• GO-SLAM	Hệ đầu tiên trong dòng NeRF có đóng vòng và tối ưu toàn cục thật sự. Nó làm được vì bản đồ neo vào khung hình chốt, nên sửa tư thế thì kéo bản đồ theo.
• vMAP	Mỗi vật thể một mạng nhỏ riêng, hợp lại lúc dựng ảnh. Vật quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật.
••• Vì sao 3DGS nhanh hơn NeRF về bản chất?	NeRF phải đi tìm vật chất bằng cách lấy hàng chục mẫu dọc mỗi tia. 3DGS đã lưu vật chất thành nguyên thể tường minh nên chỉ chiếu, sắp theo độ sâu rồi trộn. Chi phí mỗi điểm ảnh đổi từ số mẫu sang số nguyên thể phủ lên nó.
••• Ba hệ quả của việc chuyển sang nguyên thể tường minh	Cập nhật bản đồ thành sửa chữa cục bộ; bản đồ dịch chuyển được sau khi đóng vòng; đạo hàm theo tư thế có dạng giải tích.
••• Ghép chặt và ghép lỏng	Ghép chặt lấy tư thế từ chính bộ dựng ảnh, nên lỗi bản đồ chảy vào quỹ đạo. Ghép lỏng để tư thế cho đường cổ điển và coi bản đồ là sản phẩm; nó không thể làm quỹ đạo xấu đi.
•• Photo-SLAM	Giữ xương sống đặc trưng và hiệu chỉnh chùm tia cho tư thế, gắn thêm bản đồ Gaussian chỉ để dựng ảnh. Đây là mẫu ghép lỏng điển hình và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm.
••• PSNR cao nhưng ATE tệ — giải thích thế nào?	Hàm mất mát chỉ ràng buộc ảnh dựng, không ràng buộc hình học, và màu phụ thuộc góc nhìn cho mô hình chỗ để “vẽ” thay vì “dựng”. Hai đại lượng đó tách rời được hoàn toàn.
••• PSNR đo trên khung hình đầu vào	Dấu hiệu cảnh báo quan trọng nhất khi đọc bảng: con số đó đo độ khớp chứ không đo khả năng tổng quát sang góc nhìn mới. Phải tìm xem bài có tập góc nhìn giữ lại không.
••• ATE có cần chỉnh tỉ lệ	Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh cả tỉ lệ. Phép căn chỉnh đó hấp thụ sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất, nên không so trực tiếp được với hệ stereo-inertial.
•• Thí nghiệm oracle tư thế	Cấp tư thế chân trị và chỉ để hệ dựng bản đồ, để lấy trần chất lượng bản đồ khi khâu tư thế hoàn hảo. Trần đó không đủ thì mọi công sức về sau bị chặn trên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Ngân sách của một bản đồ Gaussian	Gần sáu chục số thực cho mỗi hạt, nhân bốn khi đang tối ưu vì còn gradient và hai ô Adam. Lưu lượng mỗi giây tỉ lệ với số hạt nhân số vòng tối ưu mỗi khung nhân số khung mỗi giây — nên bằng thông cần trước dung lượng, và thừa số duy nhất bạn điều khiển được là số hạt.
●● Vì sao TensorRT không cứu được?	Vì cái chạy trên Orin không phải suy luận mà là vòng lặp tối ưu trên chính tham số bản đồ. Lượng tử hoá phá gradient, đồ thị đổi liên tục, và nút thất nằm ở bằng thông chứ không ở phép nhân.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
gaussian-splatting	Bộ rasterize Gaussian khác biệt được — hạ tầng mà toàn bộ dòng SLAM Gaussian trong chương dựng lên trên đó	Hạ tầng; bền hơn tên từng hệ SLAM
Photo-SLAM [16]	Mẫu ghép lỏng: tư thế do đặc trưng và hiệu chỉnh chùm tia cổ điển, bản đồ Gaussian chỉ tiêu thụ tư thế	Kiến trúc đáng bất chước nhất cho hệ đang chạy sản phẩm
GO-SLAM [8]	Neo bản đồ neural vào khung hình chốt để biến dạng được sau khi đóng vòng	Bằng chứng cho tiêu chí “sửa được sau khi đóng vòng”
NeRF-SLAM [10]	Dùng bất định của độ sâu làm trọng số cho hàm mất mát dựng bản đồ	Tư duy ma trận thông tin mang sang phía bản đồ
Công cụ số hoá công trình và quét nội thất	Dựng lại chất lượng cao <i>ngoại tuyến</i> từ video quay tay — đúng chỗ dòng này thắng rõ	Được cho là đã dùng trong sản phẩm thương mại; tôi không kiểm chứng chi tiết

MINI PROJECT SLAM · Đặt một hệ bản đồ neural lên đúng hai trục đánh giá · 1–2 ngày

Mục tiêu	trả lời bằng số cho câu hỏi trung tâm của chương: một hệ SLAM Gaussian đứng ở góc phần tư nào của Hình 1.6 khi đo bằng <i>giao thức của bạn</i> , chứ không phải giao thức của bài báo.
Dữ liệu	một chuỗi TUM RGB-D để hệ chạy được, cộng đúng một chuỗi EuRoC khó mà bạn đã có sẵn nhiều từ dự án chương 24 — ví dụ V2_03. Baseline là hệ ORB-SLAM3 của chính bạn ở cấu hình mặc định đã tinh chỉnh.
Các bước	cài một hệ ghép chặt và chạy nó ba lần trên mỗi chuỗi; ghi ATE bằng <i>evo</i> với <i>cùng một</i> phép căn chỉnh cho cả hai hệ, và ghi rõ có căn chỉnh tỉ lệ hay không; ghi PSNR trên các khung hình giữ lại, không phải trên khung hình đầu vào; chạy thêm chế độ oracle với tư thế chân trị để lấy trần chất lượng bản đồ; đo đỉnh bộ nhớ GPU và thời gian mỗi khung hình, trên máy để bàn trước, rồi trên Orin nếu nó chạy nổi.
Xong khi	bạn vẽ được một điểm cho mỗi hệ trên mặt phẳng (chất lượng ảnh, ATE) kèm thanh sai số ba lần chạy, và phát biểu được một trong hai câu: “hệ neural nằm cùng góc với ORB-SLAM3 theo trục dọc” hoặc “nó tệ hơn X cm, tức là ngoài sàn nhiều đã đo”. Kiểm tra chéo bắt buộc: nếu bỏ căn chỉnh tỉ lệ mà ATE của hệ đơn ảnh nhảy vọt, bạn vừa nhìn thấy chỗ mà bảng công bố đã giấu.
Nối với	E/24 cho sàn nhiều và cho thí nghiệm oracle; F/27 cho khung bốn cổng để kết luận đi tiếp hay dừng; F/34 nếu chuỗi khó của bạn hỏng vì ảnh mờ chứ không vì bản đồ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bản đồ dày cho bạn ba thứ mà bản đồ thưa không có: biết chỗ nào trống để tránh vật cản, biết hình dạng bề mặt để thao tác hoặc kiểm tra, và dựng được ảnh ở góc nhìn mới. Cả ba đều là câu hỏi về *khoảng không giữa các điểm*, thứ mà bản đồ thưa chưa bao giờ mô hình hoá.
- A2. Không có lý do mặc định nào cả — và đó chính là cổng đầu tiên. Bản đồ neural chỉ hơn TSDF ở ba chỗ: nội suy vùng chưa quan sát, dựng được ảnh có màu phụ thuộc góc nhìn, và dồn tham số theo nội dung. Nếu nhu cầu của bạn không nằm trong ba chỗ đó thì TSDF hoặc lưới chiếm dụng vẫn là câu trả lời đúng.
- A3. Vì hàm mất mát chỉ ràng buộc ảnh dựng ra. Hai cấu hình hình học rất khác nhau có thể sinh ra cùng một bộ ảnh, nhất là khi màu được phép phụ thuộc hướng nhìn. Muốn biết quỹ đạo, phải đo ATE bằng đúngng giao thức căn chỉnh mà bạn dùng cho hệ hiện tại.
- A4. Thay đổi duy nhất là bỏ việc đi dò dọc tia, thay bằng chiếu các nguyên thể tường minh rồi trộn. Nó mở ra đóng vòng vì nguyên thể có vị trí tường minh: gắn chúng vào khung hình chốt thì sửa tư thế sẽ kéo được cả cụm theo, còn với một MLP đơn thì không có phép nào tương đương.
- A5. Bảng thông cần trước dung lượng, và lý do nằm ở công thức chứ không ở một con số: mỗi vòng tối ưu đọc rồi ghi lại *toàn bộ* tham số cùng gradient và hai ô trạng thái Adam, và lưu lượng đó còn phải nhân với số vòng mỗi khung rồi nhân với số khung mỗi giây. Với một bản đồ cỡ căn phòng, riêng việc di chuyển tham số đã ăn phần lớn băng thông của một máy nhúng trước khi tính một phép nhân nào. Muốn biết hệt bao nhiêu lần thì phải đo trên chính thiết bị của mình.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về hai cách ghi lại một căn phòng. Cách thứ nhất là đóng vài chục cái đinh lên tường ở những chỗ dễ nhận ra, rồi ghi lại khoảng cách giữa chúng. Cách thứ hai là thuê một họa sĩ vẽ căn phòng đó thật giống, giống tới mức nhìn bức tranh cứ tưởng đang nhìn phòng thật. Cái đinh giúp bạn biết mình đang đứng ở đâu, chính xác tới từng centimét. Nhưng nhìn bản đồ đinh thì không biết cái ghế có chắn lối đi hay không. Bức tranh thì ngược lại: nhìn rất rõ căn phòng, mà đo tường từ bức tranh lại sai, vì họa sĩ chỉ cần vẽ cho *trông* giống chứ không cần vẽ cho *đo* đúng. Anh ta được phép vẽ một cái bóng thay cho một cái hốc tường, và người xem không phát hiện ra. Toàn bộ chương này là chuyện có nên đổi đinh lấy tranh hay không. Câu trả lời hôm nay là: đừng đổi, hãy giữ đinh để định vị và treo thêm bức tranh nếu bạn thật sự cần nhìn. Vì bức tranh rất đắt — vẽ một căn phòng ngốn gần hết sức của cái máy tính nhỏ gắn trên robot — và vì độ giống của bức tranh không hề chứng minh nó đo đúng.

Chỗ không rút gọn được: chỗ bức tranh *có thể* đo đúng. Khi được vẽ từ rất nhiều góc nhìn phủ kín, họa sĩ hết chỗ để gian lận, và bức tranh dần trở thành phép đo thật. Không có ranh giới gọn nào nói bao nhiêu góc nhìn là đủ, và mọi cách nói đơn giản về ranh giới đó đều bỏ sót một phần ví dụ.

Một câu: *Đinh để biết mình ở đâu, tranh để biết chỗ đó trông thế nào — và tranh đẹp không có nghĩa là tường được đo đúng.*

1.8 Kết nối

C/16/03-nerf-3dgs-va-hop-nhat là tiền đề trực tiếp: cơ chế dựng ảnh thể tích, phép rasterize Gaussian, và lập luận vì sao PSNR không chứng minh hình học đều nằm ở đó, còn chương này chỉ hỏi chuyện dùng chúng làm bản đồ. F/27 là bộ lọc: khung bốn cổng ở đó chính là thứ dẫn tới kết luận “chưa đáng” của mục 1.5, và bạn nên chạy lại khung đó với số liệu của riêng mình. F/31 giải thích các bộ bám vết dày mà NeRF-SLAM và GO-SLAM mượn lại, nên đọc nó trước nếu bạn muốn hiểu vì sao hai hệ đó chính xác hơn phần còn lại của dòng. F/30 là nguồn của mọi tiên nghiệm độ sâu và pháp tuyến mà NICER-SLAM dùng để thay cảm biến, kèm theo vấn đề tỉ lệ đi cùng. F/33 tiếp nhận ý tưởng bản đồ mức đối tượng của vMAP và đẩy nó lên đồ thị cảnh. F/34 nhận lại hai chỗ chương này bỏ ngỏ: ảnh mờ do chuyển động, và việc lấy trọng số phần dư

từ bất định thay vì từ hằng số. Còn E/24 là bộ công cụ để chạy ba thí nghiệm ở mục 1.4 một cách có kỷ luật — đặc biệt là thí nghiệm oracle, thứ trả lời được nhiều nhất với chi phí ít nhất.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao khoản chênh tốc độ giữa NeRF và 3DGS lại kéo theo khả năng sửa bản đồ sau khi đóng vòng, trong khi hai chuyện đó nghe không liên quan gì tới nhau?
2. Một hệ báo PSNR cao nhất bảng và ATE tốt hơn mọi hệ neural khác. Vì sao điều đó vẫn chưa nói gì về việc nó có thay được hệ của bạn không, và bạn tìm dòng nào trong bài để biết?
3. Giả sử bạn buộc phải dùng bản đồ neural cho tránh vật cản. Đại lượng nào của TSDF hoặc lưới chiếm dụng bị mất, và bạn dựng lại nó bằng cách nào?
4. Bài học từ thô đến tinh của BARF tương đương với kỹ thuật cổ điển nào, và điều gì xảy ra nếu bạn thêm một số hạng quang trắc vào hệ mình mà quên nó?
5. Vì sao lượng tử hoá và TensorRT gần như vô dụng ở chương này, trong khi chúng là công cụ chính ở mọi chương khác của Phần F?
6. Một hệ ghép chặt bị lệch tư thế nhẹ ở khung hình thứ 200. Chuỗi sự kiện nào dẫn tới hỏng, và vì sao không có chỉ báo nào bật lên như ở hệ cổ điển?
7. Trong bốn bài học thiết kế ở cuối chương, bài nào áp được vào hệ của bạn ngay tuần này mà không cần thêm một dòng học sâu nào?

Nguồn

- [1] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [2] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [3] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [4] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [5] H. Wang, J. Wang, and L. Agapito, “Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [6] M. M. Johari, C. Carta, and F. Fleuret, “ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [7] E. Sandström, Y. Li, L. Van Gool, et al., “Point-SLAM: Dense Neural Point Cloud-based SLAM,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [8] Y. Zhang, F. Tosi, S. Mattoccia, et al., “GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [9] C.-H. Lin, W.-C. Ma, A. Torralba, et al., “BARF: Bundle-Adjusting Neural Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [10] A. Rosinol, J. J. Leonard, and L. Carlone, “NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields,” *arXiv preprint arXiv:2210.13641*, 2022.
- [11] Z. Zhu, S. Peng, V. Larsson, et al., “NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM,” in *International Conference on 3D Vision (3DV)*, 2024.
- [12] X. Kong, S. Liu, M. Taher, et al., “vMAP: Vectorised Object Mapping for Neural Field SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [13] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [14] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [15] S. Ha, J. Yeon, and H. Yu, “RGBD GS-ICP SLAM,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [16] H. Huang, L. Li, H. Cheng, et al., “Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.

- [18] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [19] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [20] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [21] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [22] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [23] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.